

JAVA COMPLETE GUIDE

[Basics](#) • [Advanced](#) • [SQL & Databases](#) • [Spring](#) • [Spring Boot](#)

[Definitions](#) • [Programs](#) • [Examples](#) • [Best Practices](#)

A Complete Reference for Java Developers

Chapter 1: Java Fundamentals

Definition: Java

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (now Oracle) in 1995. It follows the principle 'Write Once, Run Anywhere' (WORA) — Java code is compiled into bytecode which runs on the Java Virtual Machine (JVM) on any operating system.

Definition: JVM, JDK, JRE

JVM (Java Virtual Machine) is the engine that executes Java bytecode. JRE (Java Runtime Environment) = JVM + libraries needed to run Java programs. JDK (Java Development Kit) = JRE + development tools (compiler javac, debugger, etc.). To develop Java programs you need the JDK; to only run them, you need the JRE.

1.1 Data Types & Variables

Definition: Variable

A variable is a named memory location that stores a value. In Java, every variable must be declared with a specific data type before it can be used. The type determines how much memory is allocated and what operations are valid.

Definition: Primitive Data Types

Primitive types are the most basic data types built into Java. They hold actual values directly in memory (not references). Java has 8 primitive types: byte, short, int, long, float, double, char, and boolean.

Definition: Reference Types

Reference types (like String, arrays, and objects) store the memory address (reference) of the actual data, not the data itself. They are created using the 'new' keyword and live on the heap.

Type	Size	Range / Description	Default	Example
byte	8-bit	-128 to 127	0	byte b = 100;
short	16-bit	-32,768 to 32,767	0	short s = 5000;

int	32-bit	-2B to 2B (most common)	0	int i = 42;
long	64-bit	Very large integers	0L	long l = 9999L;
float	32-bit	Decimal (6-7 digits)	0.0f	float f = 3.14f;
double	64-bit	Decimal (15-16 digits)	0.0d	double d = 3.14159;
char	16-bit	Single Unicode character	'\u0000'	char c = 'A';
boolean	1-bit	true or false only	false	boolean b = true;

```
// Variable declaration and initialisation
int age = 25; // integer
double salary = 75000.50; // decimal
String name = "Alice"; // String (reference type)
boolean isActive = true; // true or false
char grade = 'A'; // single character

// final - constant (value cannot change after assignment)
final int MAX_SIZE = 100;
// MAX_SIZE = 200; // ERROR: cannot reassign a final variable

// Type casting
double pi = 3.99;
int rounded = (int) pi; // explicit cast -> 3 (truncates decimal)
int x = 5;
double y = x; // implicit widening -> 5.0 (safe, no data loss)

// var keyword (Java 10+) - type is inferred by compiler
var message = "Hello"; // compiler knows it's String
var count = 42; // compiler knows it's int
```

1.2 Operators

Definition: Operator

An operator is a symbol that performs an operation on one or more operands. Java provides arithmetic, relational, logical, assignment, bitwise, and ternary operators.

```
// — Arithmetic Operators —————
int a = 10, b = 3;
System.out.println(a + b); // 13 - Addition
System.out.println(a - b); // 7 - Subtraction
```

```

System.out.println(a * b);    // 30 - Multiplication
System.out.println(a / b);    // 3 - Integer Division (drops remainder)
System.out.println(a % b);    // 1 - Modulus (remainder)
System.out.println(a++);      // 10 - Post-increment (use then add)
System.out.println(++a);      // 12 - Pre-increment (add then use)

// — Relational Operators (return boolean) —————
System.out.println(a == b);    // false - equal to
System.out.println(a != b);    // true - not equal
System.out.println(a > b);     // true - greater than
System.out.println(a <= b);    // false - less than or equal

// — Logical Operators —————
boolean x = true, y = false;
System.out.println(x && y);     // false - AND (both must be true)
System.out.println(x || y);    // true - OR (at least one true)
System.out.println(!x);        // false - NOT (inverts)

// — Assignment Operators —————
int n = 10;
n += 5;    // n = 15  (n = n + 5)
n -= 3;    // n = 12
n *= 2;    // n = 24
n /= 4;    // n = 6
n %= 4;    // n = 2

// — Ternary Operator —————
// syntax: condition ? valueIfTrue : valueIfFalse
int age = 20;
String status = (age >= 18) ? "Adult" : "Minor";
System.out.println(status);    // Adult

```

1.3 Control Flow

Definition: Control Flow

Control flow statements control the order in which instructions are executed. Without them, code runs top-to-bottom once. Control flow allows decisions (if/switch), repetition (loops), and jumping (break/continue/return).

if / else if / else

Definition: if Statement

The if statement evaluates a boolean condition. If it is true, the block executes. else if provides additional conditions. else is the fallback if no condition is true.

```

int score = 85;

if (score >= 90) {

```

```

        System.out.println("Grade: A - Excellent!");
    } else if (score >= 80) {
        System.out.println("Grade: B - Good");
    } else if (score >= 70) {
        System.out.println("Grade: C - Average");
    } else if (score >= 60) {
        System.out.println("Grade: D - Below Average");
    } else {
        System.out.println("Grade: F - Fail");
    }
}
// Output: Grade: B - Good

```

switch Statement

Definition: switch Statement

The switch statement evaluates an expression and compares it against multiple 'case' values, jumping to the matching case. It is cleaner than chained if-else when checking the same variable against many values. Java 14+ introduced switch expressions that can return a value.

```

// Traditional switch
int day = 3;
switch (day) {
    case 1: System.out.println("Monday");    break;
    case 2: System.out.println("Tuesday");   break;
    case 3: System.out.println("Wednesday"); break;
    case 4: System.out.println("Thursday");  break;
    case 5: System.out.println("Friday");    break;
    default: System.out.println("Weekend");
}
// NOTE: 'break' is mandatory - without it, execution falls through to next case

// Switch Expression (Java 14+) - no break needed, returns a value
String dayName = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    case 4, 5 -> "Thursday or Friday";    // multiple cases
    default -> "Weekend";
};
System.out.println(dayName);    // Wednesday

```

Loops

Definition: Loop

A loop repeats a block of code multiple times. Java has four types: for (when iteration count is known), while (when condition is checked first), do-while (when block must run at least once), and for-each (for iterating arrays and collections).

```

// — for loop —————
// Syntax: for (init; condition; update) { body }
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " ");    // 1 2 3 4 5
}

// — while loop —————
// Runs as long as condition is true (checked BEFORE each iteration)
int count = 1;
while (count <= 5) {
    System.out.print(count + " "); // 1 2 3 4 5
    count++;
}

// — do-while loop —————
// Runs at least ONCE (condition checked AFTER each iteration)
int num = 10;
do {
    System.out.println(num);    // prints 10 even though 10 > 5
    num++;
} while (num < 5);

// — for-each (enhanced for) —————
// Used to iterate arrays and collections cleanly
int[] numbers = {10, 20, 30, 40, 50};
for (int n : numbers) {
    System.out.print(n + " ");    // 10 20 30 40 50
}

// — break and continue —————
for (int i = 1; i <= 10; i++) {
    if (i == 6) break;            // exit loop completely when i=6
    if (i % 2 == 0) continue;    // skip even numbers
    System.out.print(i + " ");    // 1 3 5
}

```

1.4 Arrays

Definition: Array

An array is a fixed-size, ordered collection of elements of the same data type, stored in contiguous memory. Once created, the size of an array cannot be changed. Arrays are zero-indexed (first element is at index 0).

```

// — 1D Array —————
// Declaring and creating
int[] scores = new int[5];        // 5 slots, all zero by default
int[] marks = {90, 85, 78, 92, 88}; // initialiser list

// Accessing elements
System.out.println(marks[0]);    // 90 (first element)
System.out.println(marks[4]);    // 88 (last element)

```

```

System.out.println(marks.length); // 5    (number of elements)

// Modifying
marks[2] = 95;                      // change index 2 from 78 to 95

// Iterating
for (int i = 0; i < marks.length; i++) {
    System.out.println("marks[" + i + "] = " + marks[i]);
}

// — 2D Array (Matrix) —————
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
System.out.println(matrix[1][2]); // 6    (row 1, col 2)

// Nested loop to print matrix
for (int[] row : matrix) {
    for (int val : row) {
        System.out.printf("%3d", val);
    }
    System.out.println();
}

// — Useful Arrays utility methods —————
import java.util.Arrays;
int[] arr = {5, 2, 8, 1, 9};
Arrays.sort(arr);                      // sorts in-place: [1,2,5,8,9]
System.out.println(Arrays.toString(arr)); // [1, 2, 5, 8, 9]
int idx = Arrays.binarySearch(arr, 5);   // index of value 5 -> 2
int[] copy = Arrays.copyOf(arr, 3);       // [1, 2, 5] (first 3)

```

1.5 Methods

Definition: Method

A method is a named block of code that performs a specific task. Methods promote reusability — you write the logic once and call it as many times as needed. A method can accept inputs (parameters) and optionally return an output (return value).

Definition: Method Overloading

Method overloading means defining multiple methods with the same name but different parameter lists (different number, type, or order of parameters). The compiler picks the correct version based on the arguments provided. Return type alone cannot distinguish overloaded methods.

```

// — Method Syntax —————

```

```

// accessModifier returnType methodName(parameters) { body }

public static int add(int a, int b) {
    return a + b;    // must return an int
}

public static void greet(String name) {    // void = returns nothing
    System.out.println("Hello, " + name + "!");
}

// — Method Overloading —————
public static int    multiply(int a, int b)        { return a * b; }
public static double multiply(double a, double b)  { return a * b; }
public static int    multiply(int a, int b, int c)  { return a * b * c; }

System.out.println(multiply(3, 4));           // calls int version -> 12
System.out.println(multiply(3.0, 4.5));       // calls double version -> 13.5
System.out.println(multiply(2, 3, 4));         // calls 3-param version -> 24

// — Varargs (Variable-length Arguments) —————
// Allows calling with any number of arguments
public static int sum(int... numbers) {
    int total = 0;
    for (int n : numbers) total += n;
    return total;
}
System.out.println(sum(1, 2));                // 3
System.out.println(sum(1, 2, 3, 4, 5));       // 15

// — Recursion —————
// A method that calls itself. Must have a base case to stop.
public static long factorial(int n) {
    if (n <= 1) return 1;                    // base case
    return n * factorial(n - 1);             // recursive call
}
System.out.println(factorial(5));             // 120    (5*4*3*2*1)

// — Pass by Value (Java always) —————
// Primitives: copy of value is passed; original unchanged
// Objects: copy of reference is passed; object's fields CAN change
public static void doubleIt(int x) { x *= 2; } // no effect outside
int val = 5;
doubleIt(val);
System.out.println(val);    // still 5

```


Chapter 2: Object-Oriented Programming (OOP)

Definition: Object-Oriented Programming (OOP)

OOP is a programming paradigm that organises code around objects — real-world entities that combine data (fields/attributes) and behaviour (methods). Java is a purely OOP language (except primitives). The four pillars of OOP are: Encapsulation, Inheritance, Polymorphism, and Abstraction.

2.1 Classes & Objects

Definition: Class

A class is a blueprint or template that defines the structure (fields) and behaviour (methods) of objects. It is like an architectural plan for a house — the plan itself is not a house, but you can create many houses from it.

Definition: Object

An object is an instance of a class — a concrete entity created from the class blueprint. Each object has its own copy of the class fields (state) but shares the class methods (behaviour). Objects are created using the 'new' keyword and stored in heap memory.

Definition: Constructor

A constructor is a special method called automatically when an object is created using 'new'. It initialises the object's fields. A constructor has the same name as the class and no return type. If you don't write one, Java provides a default no-argument constructor.

```
// Class definition - the blueprint
public class Employee {

    // Fields - instance variables (each object has its own copy)
    private int    id;
    private String name;
    private double salary;
    private String department;

    // No-arg constructor
    public Employee() {
        this.salary = 0.0;
    }

    // Parameterised constructor
    public Employee(int id, String name, double salary, String department) {
```

```

        this.id          = id;          // 'this' refers to the current object
        this.name        = name;
        this.salary      = salary;
        this.department  = department;
    }

    // Copy constructor
    public Employee(Employee other) {
        this.id          = other.id;
        this.name        = other.name;
        this.salary      = other.salary;
        this.department  = other.department;
    }

    // Getters (read access to private fields)
    public int    getId()          { return id; }
    public String getName()        { return name; }
    public double getSalary()      { return salary; }
    public String getDepartment() { return department; }

    // Setters (write access with validation)
    public void setName(String name) {
        if (name == null || name.isBlank()) throw new
IllegalArgumentException("Name required");
        this.name = name;
    }
    public void setSalary(double salary) {
        if (salary < 0) throw new IllegalArgumentException("Salary cannot be
negative");
        this.salary = salary;
    }

    // Method
    public void applyRaise(double percent) {
        this.salary += this.salary * (percent / 100);
    }

    // toString - called automatically when object is printed
    @Override
    public String toString() {
        return String.format("Employee{id=%d, name='%s', salary=%.2f, dept='%s'}",
            id, name, salary, department);
    }
}

// — Creating and using objects —————
Employee e1 = new Employee(101, "Alice", 70000, "Engineering");
Employee e2 = new Employee(102, "Bob", 60000, "Marketing");
Employee e3 = new Employee(e1); // copy of e1

System.out.println(e1.getName()); // Alice
e1.applyRaise(10); // 10% raise
System.out.println(e1.getSalary()); // 77000.0
System.out.println(e1); // calls toString()

```

2.2 Inheritance

Definition: Inheritance

Inheritance is an OOP mechanism where one class (child/subclass) acquires the properties and behaviours of another class (parent/superclass) using the 'extends' keyword. It promotes code reuse — the child gets all non-private members of the parent for free, and can add its own or override inherited ones. Java supports single inheritance (one parent per class) but a class can implement multiple interfaces.

Definition: Method Overriding

Method overriding occurs when a subclass provides a specific implementation for a method already defined in the parent class. The method in the child must have the same name, return type, and parameters as the parent's. The `@Override` annotation signals this to the compiler.

```
// Parent (Superclass)
public class Animal {
    protected String name;
    protected int    age;

    public Animal(String name, int age) {
        this.name = name;
        this.age  = age;
    }

    public void eat() { System.out.println(name + " is eating."); }
    public void sleep(){ System.out.println(name + " is sleeping."); }

    public void makeSound() {
        System.out.println(name + " makes a generic sound.");
    }

    @Override
    public String toString() { return "Animal{name='" + name + "', age=" + age +
    "'}"; }
}

// Child (Subclass) - inherits from Animal
public class Dog extends Animal {
    private String breed;

    public Dog(String name, int age, String breed) {
        super(name, age);    // MUST call parent constructor first
        this.breed = breed;
    }

    // Overriding parent method - Dog has its own sound
    @Override
    public void makeSound() {
        System.out.println(name + " barks: Woof! Woof!");
    }
}
```

```

// Dog-specific method
public void fetch() { System.out.println(name + " fetches the ball!"); }

@Override
public String toString() {
    return "Dog{name='" + name + "', breed='" + breed + "'}";
}
}

// Another child class
public class Cat extends Animal {
    public Cat(String name, int age) { super(name, age); }

    @Override
    public void makeSound() { System.out.println(name + " meows: Mrrrow!"); }

    public void purr() { System.out.println(name + " is purring..."); }
}

// — Usage —————
Dog dog = new Dog("Rex", 3, "Labrador");
Cat cat = new Cat("Whiskers", 5);

dog.eat(); // inherited: Rex is eating.
dog.makeSound(); // overridden: Rex barks: Woof! Woof!
dog.fetch(); // Dog-specific: Rex fetches the ball!
cat.makeSound(); // overridden: Whiskers meows: Mrrrow!

```

2.3 Polymorphism

Definition: Polymorphism

Polymorphism means 'many forms'. In Java, the same method call can behave differently depending on the actual object type at runtime. There are two types: Compile-time polymorphism (method overloading — resolved by the compiler) and Runtime polymorphism (method overriding — resolved by the JVM at runtime). This allows a parent class reference to point to any child class object.

```

// Runtime Polymorphism - parent reference holds child objects
Animal a1 = new Dog("Rex", 3, "Lab"); // Dog object, Animal reference
Animal a2 = new Cat("Luna", 2); // Cat object, Animal reference

// Same method call, different behaviour - decided at RUNTIME
a1.makeSound(); // Rex barks: Woof! Woof! (Dog's version)
a2.makeSound(); // Luna meows: Mrrrow! (Cat's version)

// Polymorphism with arrays - process different types uniformly
Animal[] animals = { new Dog("Rex", 3, "Lab"), new Cat("Luna", 2), new
Dog("Max", 5, "Poodle") };
for (Animal a : animals) {

```

```

        a.makeSound();    // correct version called for each object type
    }

    // instanceof - check the actual type at runtime
    for (Animal a : animals) {
        if (a instanceof Dog) {
            Dog d = (Dog) a;    // downcasting - convert Animal ref back to Dog
            d.fetch();          // can now call Dog-specific methods
        }
    }

    // Pattern matching instanceof (Java 16+) - cleaner
    for (Animal a : animals) {
        if (a instanceof Dog d) { // cast and bind in one step
            d.fetch();
        } else if (a instanceof Cat c) {
            c.purr();
        }
    }
}

```

2.4 Abstraction

Definition: Abstraction

Abstraction means hiding internal implementation details and showing only the essential features of an object. It reduces complexity — users of a class only need to know **WHAT** it does, not **HOW** it does it. Java implements abstraction through abstract classes and interfaces.

Definition: Abstract Class

An abstract class is a class that cannot be instantiated directly. It may contain abstract methods (declared but not implemented — subclasses must implement them) and concrete methods (fully implemented). Use it when you want to provide a common base with some shared implementation.

Definition: Interface

An interface is a contract that defines what methods a class must implement, without specifying how. A class can implement multiple interfaces. Since Java 8, interfaces can also have default and static methods with implementations. Since Java 9, they can have private methods.

```

// — Abstract Class —————
public abstract class Shape {
    protected String color;

    public Shape(String color) { this.color = color; }
}

```

```

    // Abstract methods - subclasses MUST implement these
    public abstract double area();
    public abstract double perimeter();

    // Concrete method - inherited as-is
    public void displayInfo() {
        System.out.printf("Shape: %s | Color: %s | Area: %.2f%n",
            getClass().getSimpleName(), color, area());
    }
}

public class Circle extends Shape {
    private double radius;
    public Circle(String color, double radius) { super(color); this.radius =
radius; }
    @Override public double area()      { return Math.PI * radius * radius; }
    @Override public double perimeter() { return 2 * Math.PI * radius; }
}

public class Rectangle extends Shape {
    private double width, height;
    public Rectangle(String color, double w, double h) { super(color); width=w;
height=h; }
    @Override public double area()      { return width * height; }
    @Override public double perimeter() { return 2 * (width + height); }
}

// Shape s = new Shape("red"); // ERROR: cannot instantiate abstract class
Shape c = new Circle("Red", 5);
Shape r = new Rectangle("Blue", 4, 6);
c.displayInfo(); // Shape: Circle | Color: Red | Area: 78.54
r.displayInfo(); // Shape: Rectangle | Color: Blue | Area: 24.00

// — Interface —————
public interface Drawable {
    void draw(); // abstract
    default void drawWithBorder() { System.out.println("Border drawn"); draw(); }
// default
    static void info() { System.out.println("Drawable interface v1.0"); }
// static
}

public interface Resizable {
    void resize(double factor);
}

// A class implementing MULTIPLE interfaces
public class Square extends Shape implements Drawable, Resizable {
    private double side;
    public Square(String color, double side) { super(color); this.side = side; }
    @Override public double area()      { return side * side; }
    @Override public double perimeter() { return 4 * side; }
    @Override public void draw()      { System.out.println("Drawing square"); }
    @Override public void resize(double factor) { this.side *= factor; }
}

```

2.5 Encapsulation

Definition: Encapsulation

Encapsulation bundles data (fields) and the methods that operate on that data into a single unit (class), and restricts direct access to some of the object's components. This is achieved by declaring fields as private and providing public getter/setter methods. Encapsulation protects data integrity and hides implementation details.

Modifier	Same Class	Same Package	Subclass	Everywhere
private	☒ Yes	☒ No	☒ No	☒ No
(default)	☒ Yes	☒ Yes	☒ No	☒ No
protected	☒ Yes	☒ Yes	☒ Yes	☒ No
public	☒ Yes	☒ Yes	☒ Yes	☒ Yes

2.6 static, final, this, super

Definition: static keyword

The static keyword means the member belongs to the CLASS, not to any particular object. Static fields are shared across all instances. Static methods can be called without creating an object. Static blocks run once when the class is loaded.

Definition: final keyword

final has three uses: (1) final variable = constant, cannot be reassigned; (2) final method = cannot be overridden by subclasses; (3) final class = cannot be subclassed (e.g., String is final).

```
public class BankAccount {
    // static field - shared by ALL BankAccount objects
    private static int    totalAccounts = 0;
    private static double interestRate  = 3.5; // same rate for all

    // instance fields - each object has its own
    private final int     accountNumber; // final: set once, never changed
    private      String   owner;
    private      double   balance;

    // static block - runs once when class is first loaded
```

```

static {
    System.out.println("BankAccount class loaded");
}

public BankAccount(String owner, double initialBalance) {
    totalAccounts++; // update shared counter
    this.accountNumber = totalAccounts; // unique sequential ID
    this.owner = owner;
    this.balance = initialBalance;
}

// static method - call on class, not object
public static int    getTotalAccounts() { return totalAccounts; }
public static double getInterestRate()  { return interestRate; }
public static void   setInterestRate(double rate) { interestRate = rate; }

public void applyInterest() {
    balance += balance * (interestRate / 100);
}
}

// Usage
BankAccount a1 = new BankAccount("Alice", 5000);
BankAccount a2 = new BankAccount("Bob", 3000);

System.out.println(BankAccount.getTotalAccounts()); // 2 - called on class
BankAccount.setInterestRate(4.0); // affects all accounts
a1.applyInterest(); // 5000 -> 5200

// final class - cannot extend String, Integer, etc.
// final method - cannot override
public final class MathConstants {
    public static final double PI = 3.14159265358979;
    public static final double E  = 2.71828182845904;
    private MathConstants() {} // prevent instantiation
}

```


Chapter 3: Advanced Java Concepts

3.1 Exception Handling

Definition: Exception

An exception is an unexpected event that disrupts the normal flow of a program (e.g., dividing by zero, file not found, null pointer access). Java uses a try-catch mechanism to handle exceptions gracefully instead of crashing the program.

Definition: Checked vs Unchecked Exceptions

Checked exceptions (e.g., `IOException`, `SQLException`) must be declared or handled at compile time. Unchecked exceptions (`RuntimeException` and its subclasses, e.g., `NullPointerException`, `ArrayIndexOutOfBoundsException`) are not required to be caught — they represent programming bugs. Errors (e.g., `OutOfMemoryError`) are serious system problems not meant to be caught.

```
// — try-catch-finally —————
try {
    int[] arr = new int[5];
    arr[10] = 100;           // ArrayIndexOutOfBoundsException
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Index error: " + e.getMessage());
} catch (NullPointerException e) {
    System.out.println("Null pointer: " + e.getMessage());
} catch (Exception e) {     // catch-all for any other exception
    System.out.println("General: " + e.getMessage());
} finally {
    // Always executes - use for cleanup (close files, connections)
    System.out.println("This always runs");
}

// — Multi-catch (Java 7+) —————
try {
    // risky code
} catch (IOException | SQLException e) {
    System.out.println("IO or DB error: " + e.getMessage());
}

// — try-with-resources (Java 7+) —————
// Resources that implement AutoCloseable are closed automatically
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"));
     Connection conn = DriverManager.getConnection(url, user, pass)) {
    String line = br.readLine();
} // br.close() and conn.close() called automatically, even on exception

// — Throwing exceptions —————
public void withdraw(double amount) {
    if (amount <= 0)         throw new IllegalArgumentException("Amount must be
    positive");
}
```

```

        if (amount > this.balance) throw new IllegalStateException("Insufficient
funds");
        this.balance -= amount;
    }

    // — Custom Exception —————
    public class InsufficientFundsException extends RuntimeException {
        private final double shortfall;

        public InsufficientFundsException(double shortfall) {
            super(String.format("Insufficient funds. Short by: %.2f", shortfall));
            this.shortfall = shortfall;
        }

        public double getShortfall() { return shortfall; }
    }

    // Usage
    try {
        account.withdraw(9999);
    } catch (InsufficientFundsException e) {
        System.out.println(e.getMessage());
        System.out.println("Short by: " + e.getShortfall());
    }

```

3.2 Collections Framework

Definition: Collections Framework

The Java Collections Framework (JCF) is a unified architecture of interfaces and classes for storing and manipulating groups of objects. It provides ready-made, efficient implementations of common data structures: lists, sets, maps, queues, etc., eliminating the need to build them from scratch.

Definition: List

An ordered collection that allows duplicate elements. Elements can be accessed by their index (position). `ArrayList` is backed by an array (fast random access). `LinkedList` is backed by a doubly-linked list (fast insertion/deletion at ends).

Definition: Set

A collection that does NOT allow duplicate elements. `HashSet` is the fastest (no order). `LinkedHashSet` preserves insertion order. `TreeSet` maintains sorted order. Use `Set` when uniqueness is the requirement.

Definition: Map

A Map stores key-value pairs. Each key must be unique; values can repeat. HashMap is the most common (no ordering). LinkedHashMap preserves insertion order. TreeMap keeps keys sorted. Retrieve values using their key in O(1) average time.

```
import java.util.*;

// — ArrayList —————
List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add("Charlie");
names.add("Alice");           // duplicates allowed in List
names.add(1, "Dave");         // insert at index 1
names.remove("Bob");           // remove by value
names.remove(0);               // remove by index
System.out.println(names.get(0)); // Dave
System.out.println(names.size()); // 3
System.out.println(names.contains("Alice")); // true
Collections.sort(names);        // alphabetical sort

// — LinkedList as Queue (FIFO) —————
Queue<String> queue = new LinkedList<>();
queue.offer("First"); // add to tail
queue.offer("Second");
queue.offer("Third");
System.out.println(queue.peek()); // First - view without removing
System.out.println(queue.poll()); // First - remove and return
System.out.println(queue.size()); // 2

// — Stack (LIFO) —————
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10); // push onto top
stack.push(20);
stack.push(30);
System.out.println(stack.peek()); // 30 - view top
System.out.println(stack.pop()); // 30 - remove top

// — HashSet —————
Set<String> fruits = new HashSet<>();
fruits.add("Apple"); fruits.add("Mango"); fruits.add("Apple"); // duplicate
ignored
System.out.println(fruits.size()); // 2
System.out.println(fruits.contains("Mango")); // true

// — HashMap —————
Map<String, Integer> scores = new HashMap<>();
scores.put("Alice", 95);
scores.put("Bob", 87);
scores.put("Carol", 91);
scores.put("Alice", 98); // key exists: value is updated

System.out.println(scores.get("Alice")); // 98
System.out.println(scores.getDefault("Dave", 0)); // 0 - safe default
System.out.println(scores.containsKey("Bob")); // true
```

```
// Iterate map
for (Map.Entry<String, Integer> entry : scores.entrySet()) {
    System.out.println(entry.getKey() + " scored " + entry.getValue());
}

// — PriorityQueue —————
// Elements processed by priority (min-heap by default)
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30); pq.offer(10); pq.offer(20);
System.out.println(pq.poll()); // 10 - smallest first
System.out.println(pq.poll()); // 20
```

3.3 Generics

Definition: Generics

Generics allow you to write type-safe code that works with any data type, specified as a parameter. Instead of writing a separate class for Integer list, String list, etc., you write one generic class. The type is specified at compile time, ensuring type safety and eliminating the need for casting.

```
// — Generic Class —————
// T is a type parameter - replaced with real type at compile time
public class Box<T> {
    private T value;

    public Box(T value)    { this.value = value; }
    public T getValue()    { return value; }
    public void setValue(T value) { this.value = value; }

    @Override
    public String toString() { return "Box[" + value + "]"; }
}

Box<Integer> intBox      = new Box<>(42);
Box<String>  strBox      = new Box<>("Hello");
Box<Double>  doubleBox   = new Box<>(3.14);

System.out.println(intBox.getValue()); // 42
System.out.println(strBox.getValue()); // Hello
// intBox.setValue("text"); // COMPILER ERROR - type safety!

// — Generic Method —————
public static <T extends Comparable<T>> T findMax(T a, T b, T c) {
    T max = a;
    if (b.compareTo(max) > 0) max = b;
    if (c.compareTo(max) > 0) max = c;
    return max;
}

System.out.println(findMax(10, 35, 22)); // 35
```

```

System.out.println(findMax("apple","mango","banana")); // mango

// — Generic Pair class —————
public class Pair<A, B> {
    private final A first;
    private final B second;
    public Pair(A first, B second) { this.first = first; this.second = second; }
    public A getFirst() { return first; }
    public B getSecond() { return second; }
    @Override public String toString() { return "(" + first + ", " + second + ")"; }
}

Pair<String, Integer> nameAge = new Pair<>("Alice", 30);
System.out.println(nameAge); // (Alice, 30)

// — Wildcards —————
// ? extends T - upper bound (can read, T or its subclasses)
public static double sumList(List<? extends Number> list) {
    return list.stream().mapToDouble(Number::doubleValue).sum();
}

System.out.println(sumList(List.of(1, 2, 3))); // works with Integer
System.out.println(sumList(List.of(1.5, 2.5, 3.0))); // works with Double

```

3.4 Lambda Expressions & Functional Interfaces

Definition: Lambda Expression

A lambda expression is a concise way to represent an anonymous function (a block of code with no name). It implements a functional interface in a single line without writing a full class. Syntax: (parameters) -> { body }. If body is one expression, curly braces and return are optional.

Definition: Functional Interface

A functional interface is an interface with exactly ONE abstract method. It is the target type for lambda expressions. Common built-in functional interfaces in java.util.function: Predicate<T> (test condition), Function<T,R> (transform T to R), Consumer<T> (use T, return nothing), Supplier<T> (produce T), BiFunction<T,U,R> (two inputs).

```

// — Lambda Basics —————
// Before lambdas - anonymous class (verbose)
Runnable r1 = new Runnable() {
    @Override public void run() { System.out.println("Running!"); }
};

// With lambda (concise)
Runnable r2 = () -> System.out.println("Running!");

// Lambda with parameters
Comparator<String> byLength = (a, b) -> a.length() - b.length();

```

```

// Lambda with block body
Comparator<String> byLengthVerbose = (a, b) -> {
    int lenDiff = a.length() - b.length();
    return lenDiff != 0 ? lenDiff : a.compareTo(b); // secondary sort
};

// — Built-in Functional Interfaces —————
import java.util.function.*;

// Predicate<T> - tests a condition, returns boolean
Predicate<Integer> isEven = n -> n % 2 == 0;
Predicate<String> isLong = s -> s.length() > 5;
Predicate<Integer> isLarge = n -> n > 100;
System.out.println(isEven.test(4)); // true
System.out.println(isEven.and(isLarge).test(200)); // true AND > 100
System.out.println(isEven.or(isLarge).test(3)); // false OR false = false

// Function<T,R> - transforms input T to output R
Function<String, Integer> strToLen = String::length; // method reference
Function<Integer, Integer> doubleIt = x -> x * 2;
Function<String, Integer> lenDouble = strToLen.andThen(doubleIt); // compose
System.out.println(strToLen.apply("Java")); // 4
System.out.println(lenDouble.apply("Hello")); // 10 (length 5, doubled)

// Consumer<T> - accepts T, returns nothing (side effect)
Consumer<String> printer = System.out::println;
Consumer<String> upperPrint = s -> System.out.println(s.toUpperCase());
Consumer<String> both = printer.andThen(upperPrint);
both.accept("hello"); // hello then HELLO

// Supplier<T> - takes nothing, produces T
Supplier<String> greeting = () -> "Hello, World!";
Supplier<Double> random = Math::random;
System.out.println(greeting.get()); // Hello, World!

// BiFunction<T,U,R> - two inputs, one output
BiFunction<String, Integer, String> repeat = (s, n) -> s.repeat(n);
System.out.println(repeat.apply("Ha", 3)); // HaHaHa

// — Lambdas with Collections —————
List<String> names = new
ArrayList<>(Arrays.asList("Charlie", "Alice", "Bob", "Diana"));

// Sort using lambda
names.sort((a, b) -> a.compareTo(b));
// Or using method reference
names.sort(String::compareTo);

// forEach
names.forEach(name -> System.out.println("Hello, " + name));
names.forEach(System.out::println); // shorter method reference

// removeIf
names.removeIf(name -> name.startsWith("A")); // removes Alice

```

3.5 Stream API

Definition: Stream

A Stream is a sequence of elements supporting sequential and parallel aggregate operations. Streams are not data structures — they don't store data. They process data from a source (collection, array, I/O) through a pipeline of operations. Streams are lazy: intermediate operations are not executed until a terminal operation is called.

Definition: Stream Pipeline

A stream pipeline has three parts: (1) Source — where data comes from (`list.stream()`, `Arrays.stream(arr)`); (2) Intermediate operations — transform the stream and return a new stream (`filter`, `map`, `sorted`, `distinct`, `limit`); (3) Terminal operation — produces a result and closes the stream (`collect`, `forEach`, `reduce`, `count`, `min`, `max`, `findFirst`).

```
import java.util.stream.*;
import java.util.*;

List<Integer> numbers = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
List<String> names = Arrays.asList("Alice","Bob","Charlie","Diana","Eve");

// — filter - keep elements matching condition —————
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0) // 2, 4, 6, 8, 10
    .collect(Collectors.toList());

// — map - transform each element —————
List<Integer> squares = numbers.stream()
    .map(n -> n * n) // 1,4,9,16,25,36,49,64,81,100
    .collect(Collectors.toList());

// — Chained pipeline —————
List<Integer> result = numbers.stream()
    .filter(n -> n % 2 == 0) // keep evens: 2,4,6,8,10
    .map(n -> n * n) // square: 4,16,36,64,100
    .filter(n -> n > 20) // keep > 20: 36,64,100
    .sorted(Comparator.reverseOrder()) // 100,64,36
    .collect(Collectors.toList());
System.out.println(result); // [100, 64, 36]

// — reduce - aggregate to single value —————
int sum = numbers.stream().reduce(0, Integer::sum); // 55
int product = numbers.stream().reduce(1, (a,b) -> a * b); // 3628800
System.out.println("Sum=" + sum + ", Product=" + product);

// — Terminal operations —————
long count = numbers.stream().filter(n -> n > 5).count(); // 5
Optional<Integer> max = numbers.stream().max(Integer::compareTo);
Optional<Integer> min = numbers.stream().min(Integer::compareTo);
```

```

max.ifPresent(m -> System.out.println("Max: " + m)); // 10

// findFirst & findAny
Optional<String> firstLong = names.stream()
    .filter(s -> s.length() > 4)
    .findFirst();
System.out.println(firstLong.orElse("none")); // Alice

// anyMatch, allMatch, noneMatch
boolean anyGt5 = numbers.stream().anyMatch(n -> n > 5); // true
boolean allPos = numbers.stream().allMatch(n -> n > 0); // true
boolean noneNeg = numbers.stream().noneMatch(n -> n < 0); // true

// — Collectors —————
// toList, toSet, toMap
Set<Integer> evenSet = numbers.stream().filter(n -> n % 2 == 0).collect(Collectors.toSet());
Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
// {true=[2,4,6,8,10], false=[1,3,5,7,9]}

Map<Integer, List<String>> byLength = names.stream()
    .collect(Collectors.groupingBy(String::length));
// {3=[Bob, Eve], 5=[Alice, Diana], 7=[Charlie]}

// joining
String joined = names.stream().collect(Collectors.joining(", ", "[", "]"));
System.out.println(joined); // [Alice, Bob, Charlie, Diana, Eve]

// statistics
IntSummaryStatistics stats =
    numbers.stream().mapToInt(Integer::intValue).summaryStatistics();
System.out.println(stats.getAverage()); // 5.5
System.out.println(stats.getSum()); // 55

// — parallel stream - multi-threaded processing —————
long parallelSum = numbers.parallelStream().mapToLong(Integer::longValue).sum();

```

3.6 Optional

Definition: Optional<T>

Optional is a container object that may or may not contain a non-null value. It was introduced in Java 8 to explicitly express the possibility of a missing value, replacing the use of null returns which lead to NullPointerException. Use Optional as a return type for methods that might not find a value.

```

import java.util.Optional;

// — Creating Optional —————
Optional<String> present = Optional.of("Hello"); // must be non-null

```



```

Optional<String> empty    = Optional.empty();           // no value
Optional<String> maybe    = Optional.ofNullable(null);  // null -> empty

// — Checking and getting value —————
System.out.println(present.isPresent()); // true
System.out.println(empty.isEmpty());      // true (Java 11+)
System.out.println(present.get());        // Hello (throws if empty!)

// — Safe value retrieval —————
System.out.println(empty.orElse("Default"));           // Default
System.out.println(empty.orElseGet(() -> "Computed")); // Computed
System.out.println(present.orElse("Default"));         // Hello

// orElseThrow
String val = empty.orElseThrow(() -> new RuntimeException("No value found"));

// — Transforming Optional —————
Optional<Integer> length = present.map(String::length); // Optional[5]
Optional<String> upper   = present.map(String::toUpperCase); // Optional[HELLO]
Optional<String> filtered = present.filter(s -> s.length() > 3); //
Optional[Hello]

// — ifPresent / ifPresentOrElse —————
present.ifPresent(s -> System.out.println("Value: " + s)); // Value: Hello
empty.ifPresentOrElse(
    s -> System.out.println("Found: " + s),
    () -> System.out.println("Not found") // this branch runs
);

// — Real usage pattern —————
public Optional<Employee> findByEmail(String email) {
    return employees.stream()
        .filter(e -> e.getEmail().equals(email))
        .findFirst();
}

// Calling code - no NullPointerException possible
findByEmail("alice@example.com")
    .map(Employee::getName)
    .ifPresentOrElse(
        name -> System.out.println("Found: " + name),
        () -> System.out.println("Employee not found")
    );

```

3.7 Multithreading & Concurrency

Definition: Thread

A thread is the smallest unit of execution within a process. Multithreading allows multiple threads to run concurrently within the same program, enabling parallel execution on multi-core CPUs. Each thread has its own stack but shares heap memory with other threads of the same process.

Definition: Concurrency Problem

When multiple threads share data, race conditions can occur — the outcome depends on the order threads execute. Synchronisation mechanisms (synchronized keyword, locks, concurrent collections) prevent this. Deadlock occurs when two threads wait on each other indefinitely.

```
// — Creating Threads —————
// Method 1: extend Thread
class DownloadThread extends Thread {
    private String url;
    public DownloadThread(String url) { this.url = url; }
    @Override public void run() {
        System.out.println(Thread.currentThread().getName() + " downloading " +
url);
    }
}

// Method 2: implement Runnable (preferred - allows extending other classes)
Runnable uploadTask = () -> {
    System.out.println(Thread.currentThread().getName() + " uploading...");
};

// Starting threads
new DownloadThread("https://example.com/file").start();
new Thread(uploadTask, "UploadThread-1").start();

// — ExecutorService - Thread Pool —————
// Reuses threads instead of creating new ones each time
ExecutorService executor = Executors.newFixedThreadPool(4); // 4 worker threads
for (int i = 1; i <= 10; i++) {
    final int taskId = i;
    executor.submit(() -> System.out.println("Task " + taskId + " by " +
Thread.currentThread().getName()));
}
executor.shutdown(); // stop accepting new tasks
executor.awaitTermination(10, TimeUnit.SECONDS); // wait for completion

// — Callable & Future —————
// Callable is like Runnable but returns a value
Callable<Long> computeTask = () -> {
    long sum = 0;
    for (long i = 1; i <= 1_000_000L; i++) sum += i;
    return sum;
};
Future<Long> future = executor.submit(computeTask);
// ... do other work while computation runs ...
Long result = future.get(); // blocks until result is ready
System.out.println("Sum = " + result); // 500000500000

// — synchronized - prevent race conditions —————
public class Counter {
    private int count = 0;

    // synchronized: only one thread can execute this at a time
```

```

    public synchronized void increment() { count++; }
    public synchronized int getCount() { return count; }
}

// — CompletableFuture - async pipeline —————
CompletableFuture<String> future2 = CompletableFuture
    .supplyAsync(() -> fetchUserData(userId)) // async step 1
    .thenApply(data -> processData(data)) // step 2 after step 1
    .thenApply(result -> formatResult(result)) // step 3
    .exceptionally(ex -> "Error: " + ex.getMessage()); // error handling

String finalResult = future2.get(); // wait for chain to complete

// — Thread-safe Collections —————
List<String> safeList = Collections.synchronizedList(new ArrayList<>());
Map<String, Integer> safeMap = new ConcurrentHashMap<>();
BlockingQueue<String> queue = new LinkedBlockingQueue<>(100);

```

3.8 String Handling

Definition: String Immutability

Strings in Java are immutable — once created, the characters cannot be changed. Any string operation (like concatenation, replace) creates a new String object. Strings are stored in the String Pool (in the heap). This is why 'abc' == 'abc' is true (same pool object) but new String('abc') == 'abc' is false (different object).

Definition: StringBuilder vs StringBuffer

StringBuilder is a mutable sequence of characters — use it when you need to build a string through many operations (avoids creating many intermediate String objects). It is faster than StringBuffer but not thread-safe. StringBuffer is thread-safe but slower. For single-threaded string building, always prefer StringBuilder.

```

// — String methods —————
String s = " Hello, Java World! ";
System.out.println(s.length()); // 22
System.out.println(s.trim()); // "Hello, Java World!"
System.out.println(s.strip()); // same as trim but Unicode-aware (Java 11+)
System.out.println(s.toLowerCase()); // " hello, java world! "
System.out.println(s.toUpperCase()); // " HELLO, JAVA WORLD! "
System.out.println(s.contains("Java")); // true
System.out.println(s.startsWith(" H")); // true
System.out.println(s.endsWith("! ")); // true
System.out.println(s.indexOf("Java")); // 9 (first occurrence)
System.out.println(s.lastIndexOf("l")); // 13
System.out.println(s.substring(8, 12)); // "Java"
System.out.println(s.replace("Java", "Python")); // replaces all
System.out.println(s.replaceAll("\\s+", "_")); // regex replace

```

```

// split
String csv = "Alice,Bob,Charlie,Diana";
String[] parts = csv.split(",");
System.out.println(parts.length); // 4

// join (Java 8+)
String joined = String.join(" | ", "Alice","Bob","Charlie");
System.out.println(joined); // Alice | Bob | Charlie

// formatted strings
String formatted = String.format("Name: %-10s | Salary: %, .2f", "Alice", 75000.5);
// Name: Alice          | Salary: 75,000.50

// isBlank, isEmpty (Java 11+)
"".isEmpty(); // true
" ".isBlank(); // true (blank = empty or whitespace only)

// ———— StringBuilder ————
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 5; i++) {
    sb.append("Item ").append(i);
    if (i < 5) sb.append(", ");
}
System.out.println(sb.toString()); // Item 1, Item 2, Item 3, Item 4, Item 5

sb.insert(0, "List: "); // prepend
sb.delete(0, 6); // remove 'List: '
sb.reverse(); // reverse characters
sb.replace(0, 4, "XXXX"); // replace range

```

Chapter 4: SQL & Databases

 Definition: Database

A database is an organised collection of structured data stored electronically. It allows efficient storage, retrieval, update, and deletion of data. A Relational Database Management System (RDBMS) organises data into tables with rows and columns, where relationships between tables are established using keys. Popular RDBMS: MySQL, PostgreSQL, Oracle, SQL Server.

 Definition: SQL

SQL (Structured Query Language) is the standard language for interacting with relational databases. It is divided into: DDL (Data Definition Language — create/modify structure), DML (Data Manipulation Language — insert/update/delete data), DQL (Data Query Language — SELECT), DCL (Data Control Language — GRANT/REVOKE permissions), TCL (Transaction Control Language — COMMIT/ROLLBACK).

4.1 Core Database Concepts

Concept	Definition
Primary Key	Unique identifier for each row in a table. Cannot be NULL. Usually auto-incremented integer or UUID.
Foreign Key	A column that references the Primary Key of another table. Enforces referential integrity between tables.
Index	A data structure that speeds up data retrieval. Works like a book index. Trade-off: faster reads, slower writes.
Constraint	Rules enforced on table columns: NOT NULL, UNIQUE, CHECK (condition), DEFAULT (value).
Transaction	A group of SQL operations that execute as a single unit. Either all succeed (COMMIT) or all fail (ROLLBACK).
ACID	Properties guaranteeing reliability: Atomicity (all or nothing), Consistency (valid state), Isolation (independent), Durability (persisted).

Normalisation	Process of organising tables to reduce redundancy. 1NF (atomic values), 2NF (no partial dependency), 3NF (no transitive dependency).
Join	Operation that combines rows from two or more tables based on a related column.

4.2 DDL – Data Definition Language

Definition: DDL

DDL statements define and modify the structure of database objects (tables, indexes, views). Key commands: CREATE (create objects), ALTER (modify objects), DROP (delete objects permanently), TRUNCATE (remove all rows, keep structure). DDL changes are auto-committed.

```
-- Create a database
CREATE DATABASE company_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE company_db;

-- Create departments table
CREATE TABLE departments (
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(100) NOT NULL UNIQUE,
    location VARCHAR(100),
    budget DECIMAL(15,2) DEFAULT 0.00,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Create employees table (with foreign key)
CREATE TABLE employees (
    emp_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(20),
    salary DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    hire_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    dept_id INT,
    manager_id INT,
    CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
ON DELETE SET NULL,
    CONSTRAINT fk_manager FOREIGN KEY (manager_id) REFERENCES employees(emp_id),
    CONSTRAINT chk_salary CHECK (salary >= 0)
);

-- ALTER - modify existing table
ALTER TABLE employees ADD COLUMN date_of_birth DATE;
ALTER TABLE employees MODIFY COLUMN phone VARCHAR(15) NOT NULL;
ALTER TABLE employees DROP COLUMN date_of_birth;
```

```

ALTER TABLE employees RENAME COLUMN phone TO mobile;
ALTER TABLE employees ADD INDEX idx_dept_salary (dept_id, salary);

-- DROP
DROP TABLE IF EXISTS employees;      -- delete table permanently
DROP DATABASE IF EXISTS company_db;

-- TRUNCATE - faster than DELETE for removing all rows
TRUNCATE TABLE employees;            -- resets AUTO_INCREMENT too

```

4.3 DML – Data Manipulation Language

Definition: DML

DML statements manipulate the data stored in tables. Key commands: INSERT (add rows), UPDATE (modify rows), DELETE (remove rows). Unlike DDL, DML changes can be rolled back if within a transaction.

```

-- INSERT single row
INSERT INTO departments (dept_name, location, budget)
VALUES ('Engineering', 'Floor 3', 500000.00);

-- INSERT multiple rows
INSERT INTO departments (dept_name, location, budget) VALUES
    ('Marketing', 'Floor 1', 200000.00),
    ('Human Resources', 'Floor 2', 150000.00),
    ('Finance', 'Floor 4', 300000.00);

-- INSERT with SELECT - copy data
INSERT INTO archive_employees (emp_id, name, left_date)
SELECT emp_id, CONCAT(first_name, ' ', last_name), NOW()
FROM employees WHERE is_active = FALSE;

-- UPDATE
UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 1; -- 10% raise
UPDATE employees SET is_active = FALSE, salary = 0 WHERE emp_id = 5;

-- UPDATE with JOIN
UPDATE employees e
JOIN departments d ON e.dept_id = d.dept_id
SET e.salary = e.salary * 1.05
WHERE d.dept_name = 'Engineering';

-- DELETE specific rows
DELETE FROM employees WHERE is_active = FALSE;
DELETE FROM employees WHERE hire_date < '2010-01-01';

-- DELETE with JOIN
DELETE e FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_name = 'Closed Division';

```

4.4 SELECT – Querying Data

Definition: SELECT Statement

The SELECT statement retrieves data from one or more tables. Its clauses must appear in this order: SELECT (columns), FROM (table), JOIN (related tables), WHERE (filter rows), GROUP BY (aggregate), HAVING (filter groups), ORDER BY (sort), LIMIT (restrict count).

```
-- Basic SELECT
SELECT * FROM employees;                -- all columns
SELECT emp_id, first_name, last_name, salary FROM employees;
SELECT DISTINCT dept_id FROM employees; -- unique values only

-- Column aliases
SELECT
    emp_id                AS id,
    CONCAT(first_name, ' ', last_name) AS full_name,
    salary * 12            AS annual_salary,
    YEAR(hire_date)        AS hire_year
FROM employees;

-- WHERE - filter rows
SELECT * FROM employees
WHERE salary > 60000
    AND dept_id IN (1, 2, 3)           -- in a list
    AND first_name LIKE 'A%'          -- starts with A
    AND last_name LIKE '%son'         -- ends with son
    AND hire_date BETWEEN '2020-01-01' AND '2023-12-31'
    AND manager_id IS NOT NULL
    AND email REGEXP '^[a-z]+@company\.com$'; -- regex

-- ORDER BY
SELECT * FROM employees
ORDER BY salary DESC, last_name ASC, first_name ASC;

-- LIMIT & OFFSET (pagination)
-- Page 1: rows 1-10, Page 2: rows 11-20, Page 3: rows 21-30
SELECT * FROM employees ORDER BY emp_id LIMIT 10 OFFSET 0; -- page 1
SELECT * FROM employees ORDER BY emp_id LIMIT 10 OFFSET 10; -- page 2

-- Aggregate functions
SELECT
    COUNT(*)                AS total_employees,
    COUNT(manager_id)       AS employees_with_manager, -- NULL not counted
    SUM(salary)             AS total_payroll,
    AVG(salary)             AS avg_salary,
    MAX(salary)             AS highest_salary,
    MIN(salary)             AS lowest_salary,
    ROUND(AVG(salary), 2) AS avg_salary_rounded
FROM employees WHERE is_active = TRUE;

-- GROUP BY + HAVING
```



```

SELECT
    d.dept_name,
    COUNT(e.emp_id)    AS headcount,
    AVG(e.salary)      AS avg_salary,
    MAX(e.salary)      AS top_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
WHERE e.is_active = TRUE
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(e.emp_id) > 3           -- groups with more than 3 employees
      AND AVG(e.salary) > 55000
ORDER BY avg_salary DESC;

```

4.5 JOINS

Definition: JOIN

A JOIN combines rows from two or more tables based on a related column. INNER JOIN returns only matching rows in both tables. LEFT JOIN returns all rows from the left table plus matches from the right (NULLs for no match). RIGHT JOIN is the reverse. FULL OUTER JOIN returns all rows from both tables. SELF JOIN joins a table with itself.

```

-- INNER JOIN - rows with matching dept_id in BOTH tables
SELECT e.emp_id, e.first_name, e.last_name, e.salary, d.dept_name, d.location
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id
ORDER BY d.dept_name, e.salary DESC;

-- LEFT JOIN - ALL employees, dept info if available (NULL if not)
SELECT e.first_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
-- employees without a department will have dept_name = NULL

-- Find employees WITHOUT a department
SELECT e.first_name FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;

-- RIGHT JOIN - ALL departments, employees if any
SELECT e.first_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
-- departments with no employees will show employee columns as NULL

-- SELF JOIN - employees and their managers (same table)
SELECT
    e.first_name AS employee,
    e.salary     AS emp_salary,
    m.first_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id

```

```
ORDER BY m.first_name, e.first_name;

-- Multiple JOINS
SELECT
    e.first_name, e.salary,
    d.dept_name,
    p.project_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
JOIN emp_projects ep ON e.emp_id = ep.emp_id
JOIN projects p ON ep.proj_id = p.proj_id
WHERE p.status = 'Active';
```

4.6 Subqueries & CTEs

Definition: Subquery

A subquery is a SELECT statement nested inside another SQL statement. It can appear in WHERE, FROM, SELECT, or HAVING clauses. Correlated subqueries reference the outer query and re-execute for each row of the outer query (slower). Non-correlated subqueries execute once.

Definition: CTE (Common Table Expression)

A CTE is a named temporary result set defined with the WITH clause. It makes complex queries more readable by breaking them into named, reusable parts. Recursive CTEs can query hierarchical data (org charts, category trees).

```
-- Subquery in WHERE - employees earning above average
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;

-- Subquery in WHERE with IN - employees in specific departments
SELECT first_name FROM employees
WHERE dept_id IN (
    SELECT dept_id FROM departments WHERE location LIKE '%Floor 3%'
);

-- Correlated subquery - each employee vs their dept average
SELECT e.first_name, e.salary, e.dept_id,
    (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id) AS dept_avg
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);

-- CTE - cleaner version of above
WITH dept_averages AS (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY dept_id
```

```

),
top_earners AS (
    SELECT e.*, da.avg_sal
    FROM employees e
    JOIN dept_averages da ON e.dept_id = da.dept_id
    WHERE e.salary > da.avg_sal
)
SELECT te.first_name, te.salary, d.dept_name, te.avg_sal
FROM top_earners te
JOIN departments d ON te.dept_id = d.dept_id
ORDER BY te.salary DESC;

-- Recursive CTE - organisation chart
WITH RECURSIVE org_chart AS (
    -- Anchor: top-level employees (CEO, no manager)
    SELECT emp_id, first_name, manager_id, 1 AS level,
           CAST(first_name AS CHAR(500)) AS path
    FROM employees WHERE manager_id IS NULL

    UNION ALL

    -- Recursive: employees under each manager
    SELECT e.emp_id, e.first_name, e.manager_id, oc.level + 1,
           CONCAT(oc.path, ' > ', e.first_name)
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.emp_id
)
SELECT LPAD('', (level-1)*4, ' ') || first_name AS hierarchy, level, path
FROM org_chart ORDER BY path;

```

4.7 Indexes, Views, Stored Procedures & Transactions

Definition: Index

An index is a separate data structure (usually B-tree) that stores a sorted copy of one or more columns, allowing the database to find rows much faster without scanning the entire table. The trade-off: indexes speed up reads but slow down writes (INSERT/UPDATE/DELETE must update the index too).

Definition: View

A view is a virtual table defined by a stored SQL query. It does not store data itself — it dynamically executes the underlying query when accessed. Views simplify complex queries, enforce security (expose only certain columns), and provide a consistent interface even if the underlying tables change.

Definition: Stored Procedure

A stored procedure is a precompiled collection of SQL statements stored in the database under a name. It can accept parameters, contain logic (IF/LOOP/CURSOR), and return

results. Benefits: performance (pre-parsed), security (users execute procedure without direct table access), code reuse.

```
-- -- Indexes -----
-- Single column index
CREATE INDEX idx_salary      ON employees(salary);
CREATE UNIQUE INDEX idx_email ON employees(email);

-- Composite index (for queries filtering on both columns)
CREATE INDEX idx_dept_salary ON employees(dept_id, salary);

-- Use EXPLAIN to check if index is used
EXPLAIN SELECT * FROM employees WHERE dept_id = 1 AND salary > 50000;

-- Drop index
DROP INDEX idx_salary ON employees;

-- -- Views -----
-- Create a view - active employees with dept info
CREATE VIEW vw_active_employees AS
SELECT e.emp_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.email, e.salary, d.dept_name, d.location
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
WHERE e.is_active = TRUE;

-- Use view like a table
SELECT * FROM vw_active_employees WHERE dept_name = 'Engineering';
SELECT dept_name, AVG(salary) FROM vw_active_employees GROUP BY dept_name;
DROP VIEW vw_active_employees;

-- -- Stored Procedure -----
DELIMITER //
CREATE PROCEDURE sp_give_raise(
    IN  p_dept_id INT,
    IN  p_percent DECIMAL(5,2),
    OUT p_affected INT
)
BEGIN
    UPDATE employees
    SET salary = salary * (1 + p_percent/100)
    WHERE dept_id = p_dept_id AND is_active = TRUE;
    SET p_affected = ROW_COUNT();
END //
DELIMITER ;

-- Call procedure
CALL sp_give_raise(1, 10.0, @count);
SELECT @count AS rows_updated;

-- -- Transaction -----
-- Transfer salary between employees (atomic)
START TRANSACTION;
    UPDATE employees SET salary = salary - 5000 WHERE emp_id = 1;
```

```

UPDATE employees SET salary = salary + 5000 WHERE emp_id = 2;
-- Check if both are still valid
SELECT salary INTO @sall FROM employees WHERE emp_id = 1;
IF @sall < 0 THEN
    ROLLBACK; -- undo all changes if invalid
ELSE
    COMMIT; -- make changes permanent
END IF;

```

4.8 JDBC – Java Database Connectivity

Definition: JDBC

JDBC (Java Database Connectivity) is a Java API that provides a standard interface for connecting Java programs to relational databases. It allows executing SQL statements, retrieving results, and managing transactions from Java code. Key classes: DriverManager (create connections), Connection, PreparedStatement, ResultSet.

```

import java.sql.*;

public class EmployeeDAO {
    private static final String URL = "jdbc:mysql://localhost:3306/company_db";
    private static final String USER = "root";
    private static final String PASS = "secret";

    // — Get connection —————
    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }

    // — SELECT (Read) —————
    public List<Employee> findBySalaryRange(double min, double max) {
        List<Employee> list = new ArrayList<>();
        // PreparedStatement prevents SQL injection
        String sql = "SELECT * FROM employees WHERE salary BETWEEN ? AND ?";

        try (Connection conn = getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setDouble(1, min);
            ps.setDouble(2, max);
            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                Employee e = new Employee();
                e.setId(rs.getInt("emp_id"));
                e.setFirstName(rs.getString("first_name"));
                e.setLastName(rs.getString("last_name"));
                e.setSalary(rs.getDouble("salary"));
                list.add(e);
            }
        } catch (SQLException ex) {
            throw new RuntimeException("DB error", ex);
        }
    }
}

```

```

        return list;
    }

    // — INSERT and get generated key —————
    public int createEmployee(Employee emp) {
        String sql = "INSERT INTO employees (first_name, last_name, email, salary,
hire_date) VALUES (?, ?, ?, ?, ?)";
        try (Connection conn = getConnection();
            PreparedStatement ps = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, emp.getFirstName());
            ps.setString(2, emp.getLastName());
            ps.setString(3, emp.getEmail());
            ps.setDouble(4, emp.getSalary());
            ps.setDate(5, Date.valueOf(emp.getHireDate()));
            ps.executeUpdate();
            ResultSet keys = ps.getGeneratedKeys();
            if (keys.next()) return keys.getInt(1);
        } catch (SQLException ex) { throw new RuntimeException(ex); }
        return -1;
    }

    // — Transaction - batch insert —————
    public void batchInsert(List<Employee> employees) throws SQLException {
        String sql = "INSERT INTO employees (first_name, salary) VALUES (?, ?)";
        try (Connection conn = getConnection()) {
            conn.setAutoCommit(false); // start transaction
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                for (Employee emp : employees) {
                    ps.setString(1, emp.getFirstName());
                    ps.setDouble(2, emp.getSalary());
                    ps.addBatch(); // queue the statement
                }
                ps.executeBatch(); // execute all at once
                conn.commit(); // persist
            } catch (SQLException ex) {
                conn.rollback(); // undo all on error
                throw ex;
            }
        }
    }
}

```

Chapter 5: Spring Framework

Definition: Spring Framework

Spring is an open-source Java framework that simplifies enterprise Java development. It solves common problems: managing object creation (IoC), dependency wiring (DI), database transactions, web development, security, and more. Spring replaces boilerplate code with annotations and configuration, letting developers focus on business logic.

5.1 Core Concepts

Definition: IoC (Inversion of Control)

IoC is a design principle where the control of object creation and lifecycle is inverted — instead of your code creating objects with 'new', the Spring container creates and manages them. Your code just declares what it needs, and Spring provides it. This decouples classes from each other.

Definition: Dependency Injection (DI)

DI is the implementation mechanism of IoC. Instead of a class creating its dependencies internally, they are injected by the container from outside. This makes classes loosely coupled, easier to test (inject mocks), and more maintainable. Spring supports three injection types: Constructor (recommended), Setter, and Field.

Definition: Spring Bean

A Spring Bean is any object whose lifecycle (creation, configuration, destruction) is managed by the Spring IoC container. Beans are defined via annotations (`@Component`, `@Service`, `@Repository`, `@Controller`) or Java configuration (`@Bean`). By default, beans are Singleton scoped — one instance per container.

Annotation	Layer	Description
<code>@Component</code>	Any	Generic Spring-managed bean. Parent of all stereotypes.
<code>@Service</code>	Business Logic	Marks service classes. Semantically meaningful, no extra behaviour.
<code>@Repository</code>	Data Access	Marks DAO classes. Also translates DB exceptions to Spring <code>DataAccessException</code> .

@Controller	Web Layer	MVC controller that handles HTTP requests and returns view names.
@RestController	Web Layer	@Controller + @ResponseBody. Returns data (JSON/XML) directly, not views.
@Configuration	Config	Marks a class as a source of @Bean definitions (Java-based config).
@Bean	Config	Marks a method that produces a Spring Bean. Used inside @Configuration.

5.2 Dependency Injection in Practice

```
// — Constructor Injection (RECOMMENDED) —————
// Advantages: required deps clearly visible, immutable, easily testable
@Service
public class OrderService {
    private final PaymentService paymentService;
    private final InventoryService inventoryService;
    private final EmailService emailService;

    // @Autowired optional if only one constructor (Spring 4.3+)
    public OrderService(PaymentService paymentService,
                       InventoryService inventoryService,
                       EmailService emailService) {
        this.paymentService = paymentService;
        this.inventoryService = inventoryService;
        this.emailService = emailService;
    }

    public Order placeOrder(Cart cart, PaymentDetails payment) {
        inventoryService.reserve(cart.getItems());
        paymentService.charge(payment, cart.getTotal());
        Order order = createOrder(cart);
        emailService.sendConfirmation(order);
        return order;
    }
}

// — Field Injection (convenient but not recommended) ———
@Service
public class ProductService {
    @Autowired // Spring injects this automatically
    private ProductRepository productRepo; // cannot be final!
    // Problem: hard to test without Spring context
}

// — Setter Injection (for optional dependencies) —————
@Service
```



```

public class NotificationService {
    private SMSGateway smsGateway;

    @Autowired(required = false) // optional - no error if not found
    public void setSmsGateway(SMSGateway smsGateway) {
        this.smsGateway = smsGateway;
    }
}

// — @Qualifier - when multiple implementations exist —
public interface PaymentGateway { void process(double amount); }

@Component("stripeGateway")
public class StripeGateway implements PaymentGateway { ... }

@Component("paypalGateway")
public class PayPalGateway implements PaymentGateway { ... }

@Service
public class CheckoutService {
    public CheckoutService(@Qualifier("stripeGateway") PaymentGateway gateway) {
        this.gateway = gateway; // specifically inject Stripe
    }
}

// — Java Config @Bean —
@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:mysql://localhost/db");
        ds.setMaximumPoolSize(10);
        return ds;
    }

    @Bean
    @Primary // default when multiple implementations
    public EmailService emailService() { return new SmtEmailService(); }
}

```

5.3 Bean Scopes & Lifecycle

Definition: Bean Scope

Scope defines how many instances of a bean the container creates. Singleton (default): one instance shared across the whole application. Prototype: new instance every time it is requested. Request: one per HTTP request (web apps). Session: one per HTTP session.

```

// — Scopes —
@Component
@Scope("singleton") // default - omit this annotation for same effect

```

```

public class CacheService { ... }           // ONE instance for entire app

@Component
@Scope("prototype")    // fresh instance every time it's injected
public class ReportGenerator { ... }        // MULTIPLE instances

@Component
@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)
public class RequestContext { ... }         // ONE per HTTP request

// — Lifecycle Callbacks —————
@Component
public class DatabasePool {
    private Connection[] pool;

    @PostConstruct    // called AFTER all dependencies are injected
    public void init() {
        System.out.println("Initialising connection pool...");
        pool = new Connection[10];
        // create DB connections
    }

    @PreDestroy       // called BEFORE bean is removed from container
    public void cleanup() {
        System.out.println("Closing connections...");
        // close all connections
    }
}

// — ApplicationContext —————
// The Spring container itself
ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);
OrderService orderService = ctx.getBean(OrderService.class);
String[] beanNames = ctx.getBeanDefinitionNames(); // all registered beans

```

5.4 Spring AOP (Aspect-Oriented Programming)

Definition: AOP (Aspect-Oriented Programming)

AOP is a programming paradigm that separates cross-cutting concerns (functionality that spans multiple classes) from business logic. Examples of cross-cutting concerns: logging, security checks, transaction management, performance monitoring. Instead of duplicating this code in every method, you define it once in an Aspect and Spring weaves it in automatically.

Definition: AOP Terminology

Aspect: class containing cross-cutting logic. Advice: the action to take (Before, After, Around, AfterReturning, AfterThrowing). Pointcut: expression that specifies which methods the advice applies to. Join Point: the specific point in execution where advice is applied (method execution). Weaving: the process of linking aspects with application code.

```

@Aspect
@Component
public class ApplicationAspects {

    // — Pointcut expressions —————
    // 'execution' - most common: matches method signature
    @Pointcut("execution(* com.example.service.*(..))") // all service methods
    public void serviceMethods() {}

    @Pointcut("execution(public * com.example..*(..))")
    public void allPublicMethods() {}

    @Pointcut("@annotation(com.example.annotation.Audited)")
    public void auditedMethods() {} // methods with @Audited annotation

    // — Before advice - runs BEFORE the method —————
    @Before("serviceMethods()")
    public void logMethodStart(JoinPoint jp) {
        String method = jp.getSignature().toShortString();
        Object[] args = jp.getArgs();
        System.out.printf("[BEFORE] Calling: %s with args: %s%n", method,
Arrays.toString(args));
    }

    // — AfterReturning - runs after method SUCCESSFULLY returns —
    @AfterReturning(pointcut = "serviceMethods()", returning = "result")
    public void logResult(JoinPoint jp, Object result) {
        System.out.printf("[AFTER] %s returned: %s%n",
            jp.getSignature().getName(), result);
    }

    // — AfterThrowing - runs when method throws exception —
    @AfterThrowing(pointcut = "serviceMethods()", throwing = "ex")
    public void logException(JoinPoint jp, Exception ex) {
        System.out.printf("[ERROR] %s threw: %s%n",
            jp.getSignature().getName(), ex.getMessage());
    }

    // — Around - MOST POWERFUL: wraps entire method call —
    @Around("@annotation(com.example.annotation.Timed)")
    public Object measureExecutionTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        try {
            Object result = pjp.proceed(); // call the actual method
            long elapsed = System.currentTimeMillis() - start;
            System.out.printf("[PERF] %s took %dms%n",
                pjp.getSignature().getName(), elapsed);
            return result;
        } catch (Throwable ex) {
            System.out.println("[PERF] Method failed: " + ex.getMessage());
            throw ex;
        }
    }

    // — Around for caching —————

```

```
private Map<String, Object> cache = new ConcurrentHashMap<>();

@Around("@annotation(com.example.annotation.Cached)")
public Object cacheResult(ProceedingJoinPoint pjp) throws Throwable {
    String key = pjp.getSignature().toString() +
Arrays.toString(pjp.getArgs());
    if (cache.containsKey(key)) {
        System.out.println("[CACHE] Cache hit for: " + key);
        return cache.get(key);
    }
    Object result = pjp.proceed();
    cache.put(key, result);
    return result;
}
}
```

Chapter 6: Spring Boot

Definition: Spring Boot

Spring Boot is an opinionated extension of the Spring Framework that makes it easy to create production-ready applications with minimal configuration. It provides: auto-configuration (automatically configures Spring based on dependencies), embedded server (Tomcat/Jetty built-in — no WAR deployment needed), starter dependencies (curated dependency bundles), Actuator (built-in monitoring), and production-ready defaults.

6.1 Project Setup

Definition: @SpringBootApplication

This single annotation on the main class combines three annotations: @Configuration (mark class as config source), @EnableAutoConfiguration (let Spring Boot auto-configure based on classpath), and @ComponentScan (scan current package and sub-packages for @Component classes).

```
// Main Application class
@SpringBootApplication
public class CompanyApplication {
    public static void main(String[] args) {
        SpringApplication.run(CompanyApplication.class, args);
    }
}

// Key pom.xml dependencies
// spring-boot-starter-web           -> REST APIs, embedded Tomcat
// spring-boot-starter-data-jpa      -> JPA, Hibernate
// spring-boot-starter-validation    -> Bean validation (@NotNull etc.)
// spring-boot-starter-security      -> Spring Security
// spring-boot-starter-test          -> JUnit, Mockito, MockMvc
// spring-boot-starter-cache         -> Caching abstraction
// mysql-connector-java              -> MySQL driver
// lombok                           -> @Data, @Builder, etc.

// application.properties - full example
server.port=8080
server.servlet.context-path=/api

spring.datasource.url=jdbc:mysql://localhost:3306/company_db?useSSL=false
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD} # from env variable
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update      # create-drop | validate | none
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

```
logging.level.com.example=DEBUG
logging.level.org.springframework.web=INFO

spring.jackson.date-format=yyyy-MM-dd
spring.jackson.serialization.write-dates-as-timestamps=false
```

6.2 JPA Entity

Definition: JPA Entity

An Entity is a Java class mapped to a database table. Each instance of the entity represents a row in the table. Annotated with `@Entity`, it must have a primary key (`@Id`). JPA (Java Persistence API) is the specification; Hibernate is the default implementation in Spring Boot.

Definition: JPA Relationships

`@OneToMany`: one parent has many children (Department has many Employees).
`@ManyToOne`: many children belong to one parent. `@ManyToMany`: e.g., Employees and Projects. `@OneToOne`: e.g., Employee and EmployeeDetails. `FetchType.LAZY` loads related data only when accessed (recommended). `FetchType.EAGER` loads immediately.

```
// — Entity —————
@Entity
@Table(name = "employees", indexes = {
    @Index(name = "idx_email", columnList = "email", unique = true),
    @Index(name = "idx_dept", columnList = "dept_id")
})
@Data // Lombok: generates getters, setters, toString, equals,
hashCode
@NoArgsConstructor
@AllArgsConstructor
@Builder // Lombok: Employee.builder().name("Alice").build()
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT
    private Long id;

    @Column(name = "first_name", nullable = false, length = 50)
    private String firstName;

    @Column(nullable = false, length = 50)
    private String lastName;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(precision = 10, scale = 2)
```

```

private BigDecimal salary;

@Column(name = "hire_date", nullable = false)
private LocalDate hireDate;

@Column(name = "is_active", nullable = false)
private Boolean active = true;

// Many employees belong to one department
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "dept_id", foreignKey = @ForeignKey(name = "fk_emp_dept"))
private Department department;

// One employee can have many projects (ManyToMany)
@ManyToMany(fetch = FetchType.LAZY)
@JoinTable(
    name = "emp_projects",
    joinColumns = @JoinColumn(name = "emp_id"),
    inverseJoinColumns = @JoinColumn(name = "project_id")
)
private Set<Project> projects = new HashSet<>();

// Audit fields - auto-populated
@CreationTimestamp
@Column(name = "created_at", updatable = false)
private LocalDateTime createdAt;

@UpdateTimestamp
@Column(name = "updated_at")
private LocalDateTime updatedAt;
}

// Department entity
@Entity
@Table(name = "departments")
@Data @NoArgsConstructor @AllArgsConstructor
public class Department {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    private String location;

    // One department has many employees (inverse side)
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval =
true)
    private List<Employee> employees = new ArrayList<>();
}

```

6.3 Spring Data JPA Repository

Definition: Spring Data JPA Repository

Spring Data JPA automatically generates the implementation of database operations. By extending `JpaRepository<Entity, ID>`, you get 18+ methods for free (save, findById, findAll, delete, count, etc.). You can declare additional methods by following naming conventions, and Spring generates the SQL automatically.

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // — Method name queries - Spring generates SQL —————
    // Naming convention: findBy + FieldName + Condition
    List<Employee> findByDepartmentId(Long deptId);
    List<Employee> findByActiveTrue();
    List<Employee> findByActiveFalse();
    Optional<Employee> findByEmail(String email);
    List<Employee> findBySalaryGreaterThan(BigDecimal salary);
    List<Employee> findBySalaryBetween(BigDecimal min, BigDecimal max);
    List<Employee> findByFirstNameContainingIgnoreCase(String name);
    List<Employee> findByFirstNameStartingWith(String prefix);
    List<Employee> findByHireDateAfter(LocalDate date);
    List<Employee> findByDepartmentNameAndActiveTrue(String deptName);

    // With Pageable - returns Page<Employee>
    Page<Employee> findByDepartmentId(Long deptId, Pageable pageable);

    // With Sort
    List<Employee> findByActiveTrue(Sort sort);

    // Count
    long countByDepartmentId(Long deptId);

    // Exists
    boolean existsByEmail(String email);

    // Delete
    void deleteByActiveFalse();

    // — JPQL query —————
    @Query("SELECT e FROM Employee e WHERE e.salary > :minSal " +
            "AND e.active = true ORDER BY e.salary DESC")
    List<Employee> findHighEarners(@Param("minSal") BigDecimal minSal);

    @Query("SELECT e FROM Employee e JOIN FETCH e.department " +
            "WHERE e.department.name = :deptName")
    List<Employee> findByDeptNameWithDept(@Param("deptName") String deptName);

    // — Native SQL query —————
    @Query(value = "SELECT e.*, d.name AS dept_name " +
            "FROM employees e JOIN departments d ON e.dept_id = d.dept_id " +
            "WHERE e.salary > :minSal",
            nativeQuery = true)
    List<Object[]> findHighEarnersNative(@Param("minSal") double minSal);
}
```



```

// — Projection - return only specific fields —————
@Query("SELECT e.firstName AS firstName, e.email AS email, e.salary AS salary
" +
      "FROM Employee e WHERE e.department.id = :deptId")
List<EmployeeSummaryProjection> findSummaryByDept(@Param("deptId") Long
deptId);

// — Modifying query —————
@Modifying
@Transactional
@Query("UPDATE Employee e SET e.salary = e.salary * :factor " +
      "WHERE e.department.id = :deptId")
int applyDeptRaise(@Param("deptId") Long deptId, @Param("factor") double
factor);
}

// Projection interface
public interface EmployeeSummaryProjection {
    String getFirstName();
    String getEmail();
    BigDecimal getSalary();
}

```

6.4 Service Layer

Definition: Service Layer

The service layer contains the business logic of the application. It sits between the controller (handles HTTP) and the repository (handles DB). Services orchestrate multiple repository calls, apply business rules, handle transactions, and throw meaningful exceptions. All business logic belongs here, not in controllers or repositories.

```

@Service
@Transactional(readOnly = true)    // all methods read-only by default
public class EmployeeService {

    private final EmployeeRepository employeeRepo;
    private final DepartmentRepository deptRepo;

    // Constructor injection
    public EmployeeService(EmployeeRepository employeeRepo,
                           DepartmentRepository deptRepo) {
        this.employeeRepo = employeeRepo;
        this.deptRepo      = deptRepo;
    }

    // Read operations
    public List<Employee> getAllEmployees() {
        return employeeRepo.findAll(Sort.by("lastName").ascending());
    }
}

```

```

    public Page<Employee> getEmployeesPaged(int page, int size, String sortBy) {
        Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy));
        return employeeRepo.findAll(pageable);
    }

    public Employee getById(Long id) {
        return employeeRepo.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Employee", "id",
id));
    }

    public List<Employee> searchByName(String name) {
        return employeeRepo.findByFirstNameContainingIgnoreCase(name);
    }

    // Write operations - override readOnly with @Transactional
    @Transactional
    public Employee createEmployee(EmployeeRequest request) {
        // Business rule: email must be unique
        if (employeeRepo.existsByEmail(request.getEmail())) {
            throw new DuplicateResourceException("Email already in use: " +
request.getEmail());
        }

        Department dept = deptRepo.findById(request.getDeptId())
            .orElseThrow(() -> new ResourceNotFoundException("Department", "id",
request.getDeptId()));

        Employee employee = Employee.builder()
            .firstName(request.getFirstName())
            .lastName(request.getLastName())
            .email(request.getEmail())
            .salary(request.getSalary())
            .hireDate(LocalDate.now())
            .department(dept)
            .active(true)
            .build();

        return employeeRepo.save(employee);
    }

    @Transactional
    public Employee updateEmployee(Long id, EmployeeRequest request) {
        Employee emp = getById(id);

        // Check email change doesn't conflict
        if (!emp.getEmail().equals(request.getEmail())
            && employeeRepo.existsByEmail(request.getEmail())) {
            throw new DuplicateResourceException("Email already taken");
        }

        emp.setFirstName(request.getFirstName());
        emp.setLastName(request.getLastName());
        emp.setEmail(request.getEmail());
        emp.setSalary(request.getSalary());
    }

```

```

        if (request.getDeptId() != null) {
            Department dept = deptRepo.findById(request.getDeptId())
                .orElseThrow(() -> new
ResourceNotFoundException("Department", "id", request.getDeptId()));
            emp.setDepartment(dept);
        }
        return employeeRepo.save(emp);
    }

    @Transactional
    public void deactivateEmployee(Long id) {
        Employee emp = getById(id);
        emp.setActive(false);
        employeeRepo.save(emp);
    }
}

```

6.5 REST Controller

Definition: REST Controller

A REST controller handles incoming HTTP requests and returns responses. It maps URL paths to Java methods using annotations: `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`. Parameters come from the URL path (`@PathVariable`), query string (`@RequestParam`), or request body (`@RequestBody`). Responses are wrapped in `ResponseEntity` to control status codes and headers.

```

@RestController
@RequestMapping("/api/v1/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    // GET /api/v1/employees
    @GetMapping
    public ResponseEntity<List<Employee>> getAllEmployees() {
        return ResponseEntity.ok(employeeService.getAllEmployees());
    }

    // GET /api/v1/employees?page=0&size=10&sortBy=salary
    @GetMapping("/paged")
    public ResponseEntity<Page<Employee>> getEmployeesPaged(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "10") int size,
        @RequestParam(defaultValue = "lastName") String sortBy) {
        return ResponseEntity.ok(employeeService.getEmployeesPaged(page, size,
sortBy));
    }
}

```

```

// GET /api/v1/employees/42
@GetMapping("/{id}")
public ResponseEntity<Employee> getById(@PathVariable Long id) {
    return ResponseEntity.ok(employeeService.getById(id));
}

// GET /api/v1/employees/search?name=alice
@GetMapping("/search")
public ResponseEntity<List<Employee>> search(@RequestParam String name) {
    return ResponseEntity.ok(employeeService.searchByName(name));
}

// POST /api/v1/employees
@PostMapping
public ResponseEntity<Employee> create(@Valid @RequestBody EmployeeRequest
request) {
    Employee created = employeeService.createEmployee(request);
    // Return 201 Created with Location header
    URI location = ServletUriComponentsBuilder
        .fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(created.getId())
        .toUri();
    return ResponseEntity.created(location).body(created);
}

// PUT /api/v1/employees/42
@PutMapping("/{id}")
public ResponseEntity<Employee> update(
    @PathVariable Long id,
    @Valid @RequestBody EmployeeRequest request) {
    return ResponseEntity.ok(employeeService.updateEmployee(id, request));
}

// DELETE /api/v1/employees/42
@DeleteMapping("/{id}")
public ResponseEntity<Void> deactivate(@PathVariable Long id) {
    employeeService.deactivateEmployee(id);
    return ResponseEntity.noContent().build(); // 204
}
}

```

6.6 Validation & Exception Handling

Definition: Bean Validation

Bean Validation (JSR-380) provides annotations to validate object fields automatically. When a `@RequestBody` is annotated with `@Valid`, Spring validates the object before the method runs. If validation fails, a `MethodArgumentNotValidException` is thrown with details of all failures.

Definition: Global Exception Handler

`@ControllerAdvice` is a class that handles exceptions across ALL controllers centrally. Instead of handling exceptions in every controller method, you define `@ExceptionHandler` methods once and they apply globally. This keeps controllers clean and centralises error response formatting.

```
// — Request DTO with validation —————
public class EmployeeRequest {

    @NotBlank(message = "First name is required")
    @Size(min = 2, max = 50, message = "First name must be 2-50 characters")
    private String firstName;

    @NotBlank(message = "Last name is required")
    @Size(min = 2, max = 50)
    private String lastName;

    @NotBlank @Email(message = "Must be a valid email address")
    private String email;

    @NotNull(message = "Salary is required")
    @DecimalMin(value = "0.0", inclusive = false, message = "Salary must be positive")
    @DecimalMax(value = "99999999.99")
    private BigDecimal salary;

    @NotNull(message = "Department ID is required")
    @Positive
    private Long deptId;
}

// — Custom exception classes —————
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String resource, String field, Object value)
    {
        super(resource + " not found with " + field + " = " + value);
    }
}

public class DuplicateResourceException extends RuntimeException {
    public DuplicateResourceException(String message) { super(message); }
}

// — API Error response body —————
@Data @AllArgsConstructor
public class ApiError {
    private int status;
    private String error;
    private String message;
    private LocalDateTime timestamp;
    private Map<String, String> fieldErrors; // for validation errors
}
```

```
// — Global exception handler —————
@ControllerAdvice
public class GlobalExceptionHandler {

    // Validation errors - 400 Bad Request
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError>
handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> fieldErrors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(err ->
            fieldErrors.put(err.getField(), err.getDefaultMessage())
        );
        ApiError error = new ApiError(400, "Validation Failed",
            "Input validation failed", LocalDateTime.now(), fieldErrors);
        return ResponseEntity.badRequest().body(error);
    }

    // Resource not found - 404
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(ResourceNotFoundException ex) {
        ApiError error = new ApiError(404, "Not Found",
            ex.getMessage(), LocalDateTime.now(), null);
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    // Duplicate resource - 409 Conflict
    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ApiError> handleDuplicate(DuplicateResourceException ex)
    {
        ApiError error = new ApiError(409, "Conflict",
            ex.getMessage(), LocalDateTime.now(), null);
        return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
    }

    // Any unexpected error - 500
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleGeneral(Exception ex) {
        ApiError error = new ApiError(500, "Internal Server Error",
            "An unexpected error occurred", LocalDateTime.now(), null);
        return
        ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

6.7 Spring Security & JWT

Definition: Spring Security

Spring Security is a powerful authentication and authorisation framework for Spring applications. It intercepts every HTTP request through a filter chain, verifying the caller's identity (authentication) and permissions (authorisation). It protects against common attacks: CSRF, session fixation, clickjacking.

Definition: JWT (JSON Web Token)

JWT is a compact, self-contained token format for securely transmitting information as a JSON object. It has three parts: Header (algorithm), Payload (claims: user data, expiry), Signature (ensures token wasn't tampered). JWTs are stateless — the server doesn't store sessions. The client sends the token in the Authorization header on every request.

```
// — Security Configuration —————
@Configuration
@EnableWebSecurity
@EnableMethodSecurity    // enables @PreAuthorize on methods
public class SecurityConfig {

    private final JwtFilter jwtFilter;
    private final UserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())           // disabled for REST APIs
            .sessionManagement(sess -> sess
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // no
sessions
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()           // public
                .requestMatchers(HttpMethod.GET,
"/api/v1/**").hasAnyRole("USER", "ADMIN")
                .requestMatchers("/api/admin/**").hasRole("ADMIN")    // admin
only
                .anyRequest().authenticated()                       // all
others need auth
            )
            .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);    // strength 12
    }

    @Bean
    public AuthenticationManager authManager(AuthenticationConfiguration config)
throws Exception {
        return config.getAuthenticationManager();
    }
}

// — JWT Utility —————
@Component
public class JwtUtil {
    @Value("${jwt.secret}") private String secret;
    @Value("${jwt.expiry}") private long expiryMs;    // e.g. 86400000 = 24h
}
```

```

    private Key getKey() { return
Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret)); }

    public String generateToken(UserDetails user) {
        return Jwts.builder()
            .setSubject(user.getUsername())
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expiryMs))
            .claim("roles", user.getAuthorities().toString())
            .signWith(getKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    public String extractUsername(String token) {
        return Jwts.parserBuilder().setSigningKey(getKey()).build()
            .parseClaimsJws(token).getBody().getSubject();
    }

    public boolean isTokenValid(String token, UserDetails user) {
        String username = extractUsername(token);
        return username.equals(user.getUsername()) && !isExpired(token);
    }

    private boolean isExpired(String token) {
        return Jwts.parserBuilder().setSigningKey(getKey()).build()
            .parseClaimsJws(token).getBody().getExpiration().before(new
Date());
    }
}

// — Auth Controller —————
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @PostMapping("/login")
    public ResponseEntity<Map<String,String>> login(@RequestBody LoginRequest req)
{
        authManager.authenticate(
            new UsernamePasswordAuthenticationToken(req.getUsername(),
req.getPassword()));
        UserDetails user =
userDetailsService.loadUserByUsername(req.getUsername());
        String token = jwtUtil.generateToken(user);
        return ResponseEntity.ok(Map.of("token", token, "type", "Bearer"));
    }
}

// — Method-level security —————
@PreAuthorize("hasRole('ADMIN')")
public void deleteEmployee(Long id) { ... }

@PreAuthorize("hasRole('ADMIN') or #id == authentication.principal.id")
public Employee getEmployee(Long id) { ... } // own data or admin

```


6.8 Profiles & Configuration

Definition: Spring Profiles

Profiles allow different configurations for different environments (dev, test, staging, prod). You can have separate property files per profile (application-dev.properties, application-prod.properties). Beans can also be conditional on a profile using `@Profile`. Profiles are activated via JVM argument, environment variable, or application.properties.

```
// application.properties (common to all)
spring.application.name=company-app
app.name=Company Application
app.version=1.0.0

// application-dev.properties
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=create-drop
logging.level.com.example=DEBUG
logging.level.org.hibernate.SQL=DEBUG

// application-prod.properties
spring.datasource.url=jdbc:mysql://prod-db:3306/company
spring.datasource.username=${PROD_DB_USER}
spring.datasource.password=${PROD_DB_PASS}
spring.jpa.hibernate.ddl-auto=validate
logging.level.com.example=WARN
server.ssl.enabled=true

// Activate profile
// Option 1: JVM arg: -Dspring.profiles.active=prod
// Option 2: Env var: SPRING_PROFILES_ACTIVE=prod
// Option 3: In application.properties: spring.profiles.active=dev

// — @Profile - conditional beans —————
@Configuration
public class DataSourceConfig {
    @Bean @Profile("dev")
    public DataSource devDataSource() { return new
EmbeddedDatabaseBuilder().build(); }

    @Bean @Profile("prod")
    public DataSource prodDataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setMaximumPoolSize(50);
        return ds;
    }
}

// — @ConfigurationProperties —————
@ConfigurationProperties(prefix = "app")
@Component
@Data
public class AppProperties {
    private String name;
}
```

```

    private String version;
    private int maxUploadSizeMb = 10;
    private List<String> allowedOrigins = new ArrayList<>();
}
// application.properties:
// app.name=My App
// app.max-upload-size-mb=50
// app.allowed-origins=https://front.com,https://admin.com

```

6.9 Caching

Definition: Spring Cache

Spring's caching abstraction provides a consistent API over various caching providers (Ehcache, Redis, Caffeine). By annotating methods with `@Cacheable`, Spring stores the return value in cache and returns it directly on subsequent calls with the same arguments, avoiding repeated DB queries or expensive computations.

```

// Enable caching in main class
@SpringBootApplication
@EnableCaching
public class CompanyApplication { ... }

// — Cache annotations —————
@Service
public class EmployeeService {

    // @Cacheable - return value cached, subsequent calls return from cache
    @Cacheable(value = "employees", key = "#id",
               condition = "#id > 0",           // only cache when id > 0
               unless = "#result == null")      // don't cache null results
    public Employee getById(Long id) {
        System.out.println("Hitting DB for id: " + id); // only prints on cache
        miss
        return employeeRepo.findById(id).orElseThrow(...);
    }

    // @CachePut - always executes method AND updates cache
    @CachePut(value = "employees", key = "#result.id")
    public Employee updateEmployee(Long id, EmployeeRequest req) {
        // ... update logic
        return employeeRepo.save(emp);
    }

    // @CacheEvict - remove from cache (after delete/update)
    @CacheEvict(value = "employees", key = "#id")
    public void deleteEmployee(Long id) {
        employeeRepo.deleteById(id);
    }

    // Evict entire cache
    @CacheEvict(value = "employees", allEntries = true)

```

```

    @Scheduled(cron = "0 0 2 * * ?") // 2am daily
    public void clearCache() { }
}

// — Redis configuration —————
// application.properties
spring.cache.type=redis
spring.redis.host=localhost
spring.redis.port=6379
spring.redis.password=${REDIS_PASSWORD}
spring.cache.redis.time-to-live=600000 // 10 minutes in ms

```

6.10 Testing

Definition: Unit Testing

Unit tests test a single class or method in isolation. Dependencies are replaced with mocks (fake objects that simulate real ones). JUnit 5 provides `@Test`, `@BeforeEach`, `@AfterEach`. Mockito provides `@Mock`, `@InjectMocks`, `when().thenReturn()`, `verify()`. Tests should be fast (milliseconds) and not need a database.

Definition: Integration Testing

Integration tests test multiple components working together, including the real Spring context, database, and HTTP layer. `@SpringBootTest` loads the full application context. `@AutoConfigureMockMvc` provides `MockMvc` to simulate HTTP requests without a real server. Use `@Transactional` to rollback DB changes after each test.

```

// — Unit Test —————
@ExtendWith(MockitoExtension.class)
class EmployeeServiceTest {

    @Mock
    EmployeeRepository employeeRepo;

    @Mock
    DepartmentRepository deptRepo;

    @InjectMocks
    EmployeeService employeeService;

    @Test
    @DisplayName("getById should return employee when found")
    void getById_shouldReturnEmployee_whenExists() {
        // Given
        Employee emp = new Employee(1L, "Alice", "Smith", "alice@mail.com", ...);
        when(employeeRepo.findById(1L)).thenReturn(Optional.of(emp));

        // When
        Employee result = employeeService.getById(1L);
    }
}

```

```

        // Then
        assertThat(result).isNotNull();
        assertThat(result.getFirstName()).isEqualTo("Alice");
        assertThat(result.getEmail()).isEqualTo("alice@mail.com");
        verify(employeeRepo, times(1)).findById(1L);
    }

    @Test
    @DisplayName("getById should throw exception when employee not found")
    void getById_shouldThrowException_whenNotFound() {
        // Given
        when(employeeRepo.findById(999L)).thenReturn(Optional.empty());

        // When / Then
        assertThrows(ResourceNotFoundException.class,
            () -> employeeService.getById(999L));
        verify(employeeRepo, times(1)).findById(999L);
    }

    @Test
    void createEmployee_shouldThrow_whenEmailExists() {
        EmployeeRequest req = new
EmployeeRequest("Bob", "Jones", "existing@mail.com", ...);
        when(employeeRepo.existsByEmail("existing@mail.com")).thenReturn(true);

        assertThrows(DuplicateResourceException.class,
            () -> employeeService.createEmployee(req));
        verify(employeeRepo, never()).save(any()); // save should NOT be called
    }
}

// — Integration Test —————
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK)
@AutoConfigureMockMvc
@Transactional // each test rolls back DB changes
class EmployeeControllerIT {

    @Autowired MockMvc mockMvc;
    @Autowired ObjectMapper objectMapper;
    @Autowired EmployeeRepository employeeRepo;

    @Test
    void getEmployee_shouldReturn200_whenExists() throws Exception {
        mockMvc.perform(get("/api/v1/employees/1")
            .header("Authorization", "Bearer " + validToken)
            .contentType(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.firstName").value("Alice"))
            .andExpect(jsonPath("$.email").value("alice@mail.com"));
    }

    @Test
    void createEmployee_shouldReturn201_withValidData() throws Exception {
        EmployeeRequest req = new
EmployeeRequest("Dave", "Wilson", "dave@mail.com", ...);

```

```
        mockMvc.perform(post("/api/v1/employees")
            .header("Authorization", "Bearer " + adminToken)
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(req)))
        .andExpect(status().isCreated())
        .andExpect(header().exists("Location"))
        .andExpect(jsonPath("$.id").exists());
    }

    @Test
    void getEmployee_shouldReturn404_whenNotFound() throws Exception {
        mockMvc.perform(get("/api/v1/employees/9999")
            .header("Authorization", "Bearer " + validToken))
        .andExpect(status().isNotFound())
        .andExpect(jsonPath("$.error").value("Not Found"));
    }
}
```

Chapter 7: Design Patterns & Best Practices

Definition: Design Pattern

A design pattern is a reusable, named solution to a commonly occurring software design problem. Patterns are not code — they are templates/blueprints that can be applied in different situations. They fall into three categories: Creational (object creation), Structural (object composition), Behavioural (object communication).

7.1 Creational Patterns

Singleton Pattern

Definition: Singleton

Ensures that a class has only ONE instance throughout the application and provides a global access point to it. Use it for: database connection pools, configuration managers, logging service, thread pools. Warning: singletons are hard to unit test and can hide dependencies.

```
// Thread-safe Singleton (Bill Pugh - best approach)
public class ConfigurationManager {
    private Properties config;

    private ConfigurationManager() {
        // private constructor - prevent external instantiation
        config = new Properties();
        // load config from file...
    }

    // Static inner class loaded lazily when getInstance() is called
    private static class Holder {
        static final ConfigurationManager INSTANCE = new ConfigurationManager();
    }

    public static ConfigurationManager getInstance() {
        return Holder.INSTANCE;    // thread-safe, no synchronisation needed
    }

    public String get(String key)          { return config.getProperty(key); }
    public void  set(String key, String value) { config.setProperty(key, value); }
}

// Usage
ConfigurationManager cfg = ConfigurationManager.getInstance();
cfg.set("db.host", "localhost");
System.out.println(cfg.get("db.host"));    // localhost
```

Builder Pattern

Definition: Builder

Separates the construction of a complex object from its representation. Instead of a constructor with many parameters, you build the object step by step, setting only the properties you need. Use it when: objects have many optional fields, objects need to be immutable, object creation has multiple steps.

```
// Builder Pattern
public class HttpRequest {
    private final String      url;
    private final String      method;
    private final Map<String, String> headers;
    private final String      body;
    private final int         timeoutSeconds;

    // Private constructor - only Builder can call it
    private HttpRequest(Builder builder) {
        this.url          = builder.url;
        this.method        = builder.method;
        this.headers       = Collections.unmodifiableMap(builder.headers);
        this.body          = builder.body;
        this.timeoutSeconds = builder.timeoutSeconds;
    }

    // Getters
    public String getUrl()    { return url; }
    public String getMethod() { return method; }

    public static class Builder {
        private final String url;           // required
        private String      method = "GET"; // optional with defaults
        private Map<String, String> headers = new HashMap<>();
        private String      body;
        private int         timeoutSeconds = 30;

        public Builder(String url) { this.url = url; }
        public Builder method(String method) { this.method = method; return
this; }
        public Builder header(String k, String v) { headers.put(k, v); return
this; }
        public Builder body(String body)          { this.body = body; return this; }
        public Builder timeout(int seconds)        { timeoutSeconds = seconds; return
this; }
        public HttpRequest build()                  { return new HttpRequest(this); }
    }
}

// Usage - fluent API
HttpRequest req = new HttpRequest.Builder("https://api.example.com/employees")
    .method("POST")
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer xyz")
    .body("{\"name\":\"Alice\"}")
    .timeout(60)
```

```

        .build();

// Lombok @Builder - generates this automatically
@Builder
public class Employee {
    private String name; private String email; private double salary;
}
Employee e =
Employee.builder().name("Alice").email("a@b.com").salary(70000).build();

```

Factory Method Pattern

Definition: Factory Method

Defines an interface for creating an object, but lets subclasses or a factory method decide which class to instantiate. The Factory Pattern centralises object creation logic, making it easy to add new types without changing existing code (Open/Closed Principle).

```

// Product interface
public interface Notification {
    void send(String recipient, String message);
    String getType();
}

// Concrete products
@Component
public class EmailNotification implements Notification {
    @Override public void send(String to, String msg) {
        System.out.printf("[EMAIL] To: %s | Msg: %s\n", to, msg);
    }
    @Override public String getType() { return "EMAIL"; }
}

@Component
public class SMSNotification implements Notification {
    @Override public void send(String phone, String msg) {
        System.out.printf("[SMS] To: %s | Msg: %s\n", phone, msg);
    }
    @Override public String getType() { return "SMS"; }
}

@Component
public class PushNotification implements Notification {
    @Override public void send(String deviceId, String msg) {
        System.out.printf("[PUSH] Device: %s | Msg: %s\n", deviceId, msg);
    }
    @Override public String getType() { return "PUSH"; }
}

// Factory - Spring-friendly version
@Component
public class NotificationFactory {
    private final Map<String, Notification> registry;

```



```

// Spring injects all Notification implementations automatically
public NotificationFactory(List<Notification> notifications) {
    registry = notifications.stream()
        .collect(Collectors.toMap(Notification::getType, n -> n));
}

public Notification get(String type) {
    Notification n = registry.get(type.toUpperCase());
    if (n == null) throw new IllegalArgumentException("Unknown type: " +
type);
    return n;
}

}

// Usage
Notification email = factory.get("EMAIL");
email.send("alice@mail.com", "Welcome!");
Notification sms = factory.get("SMS");
sms.send("+91-9999999999", "OTP: 123456");

```

7.2 Behavioural Patterns

Strategy Pattern

Definition: Strategy

Defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime. The client selects which algorithm (strategy) to use without changing the context class. Use it when: you have multiple ways to do something, you want to switch algorithms at runtime, you want to avoid large if/else chains.

```

// Strategy interface
public interface SortingStrategy {
    void sort(int[] array);
    String getName();
}

// Concrete strategies
public class BubbleSort implements SortingStrategy {
    @Override public void sort(int[] arr) {
        // bubble sort implementation
        for (int i = 0; i < arr.length-1; i++)
            for (int j = 0; j < arr.length-i-1; j++)
                if (arr[j] > arr[j+1]) { int tmp=arr[j]; arr[j]=arr[j+1];
arr[j+1]=tmp; }
    }
    @Override public String getName() { return "BubbleSort"; }
}

public class QuickSort implements SortingStrategy {
    @Override public void sort(int[] arr) { Arrays.sort(arr); } // simplified
    @Override public String getName() { return "QuickSort"; }
}

```

```

}

// Context - uses a strategy
public class Sorter {
    private SortingStrategy strategy;

    public Sorter(SortingStrategy strategy) { this.strategy = strategy; }

    // Strategy can be changed at runtime
    public void setStrategy(SortingStrategy strategy) { this.strategy = strategy; }
}

    public int[] sort(int[] arr) {
        int[] copy = Arrays.copyOf(arr, arr.length);
        strategy.sort(copy);
        System.out.println("Sorted using: " + strategy.getName());
        return copy;
    }
}

// Usage
int[] data = {5, 2, 8, 1, 9, 3};
Sorter sorter = new Sorter(new BubbleSort());
sorter.sort(data);

sorter.setStrategy(new QuickSort()); // switch at runtime!
sorter.sort(data);

// With lambda - no class needed for simple strategies
sorter.setStrategy(arr -> { /* counting sort */ });

```

Observer Pattern

Definition: Observer

Defines a one-to-many dependency where multiple observers (listeners) are notified automatically when a subject (publisher) changes state. This is the foundation of event-driven programming. Java has built-in support through `java.util.EventListener`, Spring has `ApplicationEvent/ApplicationListener`.

```

// Subject (Publisher)
public class StockMarket {
    private List<StockObserver> observers = new ArrayList<>();
    private Map<String, Double> prices = new HashMap<>();

    public void addObserver(StockObserver o) { observers.add(o); }
    public void removeObserver(StockObserver o) { observers.remove(o); }

    public void updatePrice(String symbol, double price) {
        double oldPrice = prices.getDefault(symbol, 0.0);
        prices.put(symbol, price);
        notifyObservers(symbol, oldPrice, price); // notify all observers
    }
}

```

```

        private void notifyObservers(String symbol, double old, double current) {
            observers.forEach(o -> o.onPriceChange(symbol, old, current));
        }
    }

    // Observer interface
    public interface StockObserver {
        void onPriceChange(String symbol, double oldPrice, double newPrice);
    }

    // Concrete observers
    public class AlertService implements StockObserver {
        @Override
        public void onPriceChange(String symbol, double old, double current) {
            double change = ((current - old) / old) * 100;
            if (Math.abs(change) > 5) { // alert on 5%+ change
                System.out.printf("ALERT: %s changed by %.2f%%\n", symbol, change);
            }
        }
    }

    // Spring ApplicationEvent (built-in Observer)
    // Event
    public class EmployeeCreatedEvent extends ApplicationEvent {
        private final Employee employee;
        public EmployeeCreatedEvent(Object source, Employee emp) {
            super(source); this.employee = emp;
        }
        public Employee getEmployee() { return employee; }
    }

    // Publisher
    @Service
    public class EmployeeService {
        @Autowired ApplicationEventPublisher publisher;
        public Employee createEmployee(EmployeeRequest req) {
            Employee emp = employeeRepo.save(buildEmployee(req));
            publisher.publishEvent(new EmployeeCreatedEvent(this, emp)); // notify
            return emp;
        }
    }

    // Listener
    @Component
    public class WelcomeEmailListener {
        @EventListener
        public void onEmployeeCreated(EmployeeCreatedEvent event) {
            emailService.sendWelcome(event.getEmployee().getEmail());
        }
    }

```

7.3 Best Practices Summary

 **KEY POINT:** Apply SOLID principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. These are the foundation of clean, maintainable OOP code.

- ALWAYS use constructor injection over field injection — it makes dependencies explicit, allows immutable fields, and enables easy unit testing without a Spring context.
- NEVER concatenate SQL strings — ALWAYS use PreparedStatement or Spring Data query methods. SQL injection is one of the most common and dangerous vulnerabilities.
- Program to interfaces, not implementations: List<String> not ArrayList<String>, PaymentGateway not StripeGateway. This makes swapping implementations easy.
- Keep methods short and focused (Single Responsibility). If a method is longer than 20 lines, consider splitting it. One method should do one thing well.
- Use meaningful, descriptive names: calculateMonthlyPayroll() not calc(), employeeCount not ec. Code is read far more than it is written.
- Never return null from a method — use Optional<T> instead. Null is the 'billion-dollar mistake' and leads to NullPointerException.
- Always close resources (Connection, InputStream, etc.) using try-with-resources. Resource leaks cause memory exhaustion in production.
- Handle exceptions at the right level — catch exceptions where you can meaningfully handle them; let others propagate up. Never swallow exceptions silently.
- Put @Transactional on service methods, not repository or controller methods. Use readOnly=true as default and override for write operations.
- Write unit tests for every public service method. Target 80%+ code coverage. Tests serve as living documentation of expected behaviour.
- Externalise all configuration (DB credentials, API keys, server addresses) to environment variables or application.properties. Never hardcode secrets.
- Use DTOs (Data Transfer Objects) to decouple your API contract from your database entities. Entities can change without breaking the API.
- Log at appropriate levels: DEBUG for development details, INFO for business events, WARN for recoverable issues, ERROR for failures. Never log passwords or PII.
- Apply pagination to all list endpoints. Never return unlimited records — it will eventually crash the server under load.

7.4 HTTP Status Codes Reference

Code	Name	Meaning	Common Use Case
200	OK	Request succeeded	GET, PUT, PATCH response

201	Created	Resource successfully created	POST response
204	No Content	Success, no body returned	DELETE response
301	Moved Permanently	URL changed permanently	URL redirects
400	Bad Request	Invalid request syntax or data	Validation failures
401	Unauthorized	Authentication required	Missing or invalid token
403	Forbidden	Authenticated but not permitted	Role-based access
404	Not Found	Resource does not exist	Invalid ID
409	Conflict	State conflict	Duplicate email/username
422	Unprocessable Entity	Semantically invalid data	Business rule violation
429	Too Many Requests	Rate limit exceeded	API throttling
500	Internal Server Error	Server-side failure	Unhandled exception
503	Service Unavailable	Server not ready	Maintenance, overload

— End of Guide —

Keep building. Keep practising. Keep reading the official docs.

docs.spring.io • docs.oracle.com • hibernate.org